

Catan Game Project: Code Overview

Anthony van Tilburg

Introduction

This overview (version 4), crafted as of May 2026, highlights the project's core components and GUI. It marks a significant new milestone: full human-player interactivity during Initial Placement, combined with a rich selection of placement algorithms available for all AI players.

The project initializes an empty game board consisting of 45 tiles (19 land tiles and 26 sea tiles), a detailed scoreboard displaying player statistics, and an interaction panel with action buttons such as "Buy City" and "Roll Dice". The GUI is divided into three main areas:

- The **playboard**, which visually represents the hexagonal board with tiles, intersections, ports, and number chits;
- The **scoreboard**, showing each player's resources and key game statistics;
- The **button panel**, which enables the human player to interact with the game.

Versions 1 and 2 focused on establishing the visual foundation and core game logic. Version 3 introduced full human interactivity during Initial Placement, while the current version (v4) expands this further by implementing multiple placement algorithms for AI players.

Core Modules

The project's structure, as outlined in Annex 1, is organized into directories that separate concerns effectively.

Core

The core/ directory houses the essential game logic, comprising four key modules:

- board.py,
- player.py
- game.py
- constants.py

Each module plays a critical role in initializing and managing the game's foundational elements.

The board.py module is responsible for creating and managing the game board. It defines the layout of 45 tiles (19 land + 26 sea tiles, along with intersections and roads). The board can be initialized with a predefined configuration or randomly generated, ensuring flexibility for different game setups. It also handles port placement.

The player.py module manages player data, including resources (wheat, ore, wood, brick and sheep), development cards, and game statistics like victory points. It supports both human and AI players, with attributes initialized to reflect an empty starting state. This module is pivotal for tracking player progress, which is displayed on the scoreboard.

The game.py module serves as the central hub, orchestrating game state, player management, and board interactions. It initializes the game with a sequence number, ID and phase (e.g., "Initial Placement"), setting up players and the board. It also includes supporting classes for tracking turn details and resource distribution, ensuring all game elements are synchronized.

The constants.py module defines game-wide constants, such as resource costs (e.g., 1 wheat, 1 wood, 1 brick, 1 wool for a settlement) and log file paths. It also provides utility functions for calculations, ensuring consistency across modules.

Gui

The gui/ directory houses the essential GUI logic, now comprising five key modules:

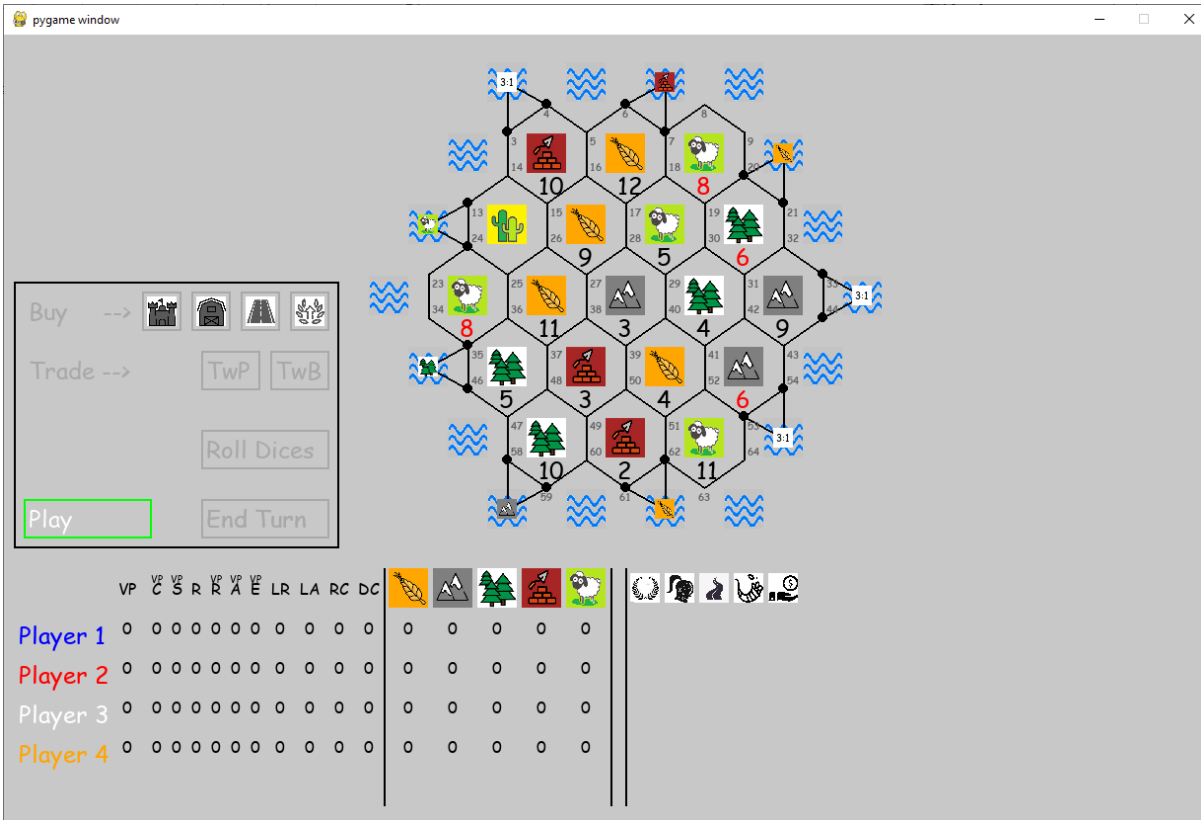
- gui.py (playboard rendering, animation queue, scoreboard, and general GUI management)
- gui_guidance.py (state machine for human placement)
- gui_human_player.py (button rendering and mode handling)
- gui_constants.py (positioning constants, fonts, colours, images, sounds)
- event_handler.py (mouse-click processing and routing)

Each module plays a critical role in rendering the game and enabling smooth human interaction.

Module	Description
GUI	Manages board rendering, the central animation queue, scoreboard updates, and overall GUI orchestration.
HumanGuidance	Implements the complete state machine for human initial placement (CHOOSING_SETTLEMENT → SETTLEMENT_SELECTED → CHOOSING_ROAD → ROAD_SELECTED), including pulsing highlights and on-screen OKY/OKN confirmation icons.
GUIHumanPlayer	Handles rendering of all action buttons (“Buy City”, “Roll Dice”, “Play”, etc.) and their active/inactive states based on game phase and current mode.
GUI Constants	Centralizes all positioning constants, fonts, colours, images, and sounds used throughout the GUI.
EventHandler	Processes every mouse click and routes it correctly (first to confirmation icons, then to the “Play” button, then to HumanGuidance).

Note: gui_game.py functionality (in version 1) has been integrated directly into gui.py for streamlined updates.

These core modules form the foundation of the game’s logic, enabling the GUI to visualize the initial state effectively; example of the gui down below.



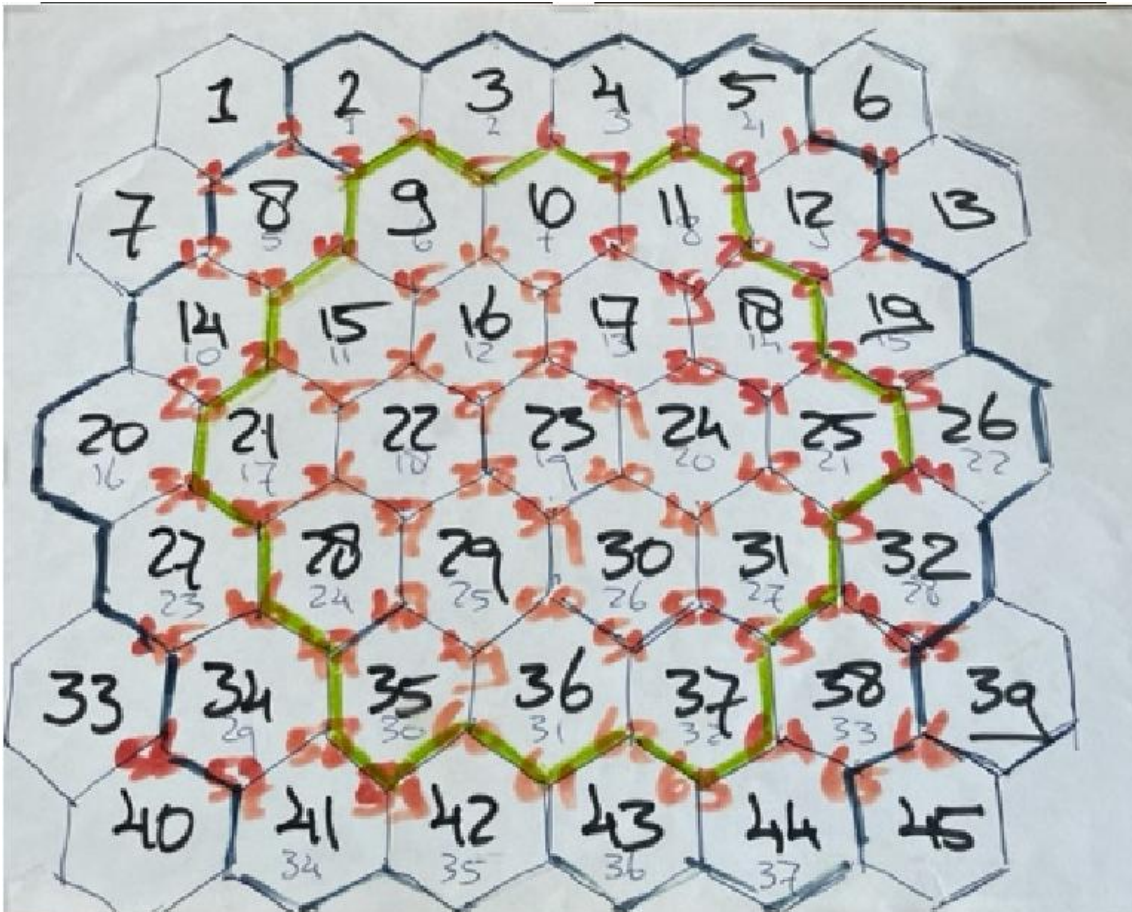
Core Classes

The project's functionality is encapsulated in 20 classes, as listed in Annex2, which define the game's entities and their interactions. Below, we highlight the core classes critical to this first version, focusing on their roles in initializing the game and supporting the GUI.

Core

The Game class (game.py) is the central orchestrator, managing the game's state, including the phase ("Initial Placement"), turn, and player list. It initializes the board and players, setting up the initial empty state. Supporting classes like StrategyDashboard, TurnDetails, ResourceCardDashboard, and Settings track player statistics, turn-specific details, resource distribution, and game configurations, respectively. These classes ensure that the game state is robust and ready for future gameplay mechanics.

The Board class (board.py) defines the game board, managing 45 tiles through the Tile class, 66 intersections via the Intersection class, and 68 roads via the Road class. The Tile class handles tile properties (e.g. type,value), Intersection tracks vertex states (e.g., occupancy, port status), and Road manages edge connections between Tiles. These classes enable the playboard's visual representation ensuring proper setup.



- Tiles: 0, 1, 6, 7, 13, 33, 39, 40, 45 have type None.
- Intersections: 0, 1, 2, 10, 11, 12, 22, 45, 55, 56, 57, 65, 66 have canbuild_ynx = "X"
- Roads: (1,2), (2,3), (9,10), (10,11), (12,13), (21,22), (45,46), (54,55), (56,57), (57,58), (64,65), (65,66) have type None.

The Player class (player.py) represents a player, managing resources, development cards, settlements, cities, and roads. It initializes with zero resources and tracks statistics like victory points, which are displayed on the scoreboard. The ResourceAsResourceCard enum (constants.py) defines resource types (e.g., WHEAT, ORE), ensuring consistent resource management.

Gui

The HumanGuidance class (gui_guidance.py) – state controlling, pulsing highlights, and confirmation icons. Supported state changes:

- CHOOSING_SETTLEMENT → SETTLEMENT_SELECTED (via on_board_click after valid intersection click)
- SETTLEMENT_SELECTED → CHOOSING_ROAD (via on_confirmation OKY: settlement placed)
- CHOOSING_ROAD → ROAD_SELECTED (via on_board_click after valid road click)
- ROAD_SELECTED → IDLE (via on_confirmation OKY + _finish_human_turn: road placed + "Play" button enabled)

The EventHandler class (event_handler.py) – processes all mouse clicks and delegates to guidance or the "Play" button.

The GUI class (gui.py) - central rendering engine that draws the static board base, all permanent settlements/roads/cities, manages the animation queue for pulsing highlights and quarter-circle reveal

animations, updates the scoreboard and round/turn display, handles screenshots, and integrates the HumanGuidance system for interactive initial placement.

The GUIHumanPlayer class (`gui_human_player.py`) – renders and controls all interaction-panel buttons (Buy City, Buy Settlement, Buy Road, Buy DCard, TwP, TwB, Roll Dices, End Turn, Cancel, and the key “Play” button) with dynamic active/inactive states based on game phase and current HumanGuidance state.

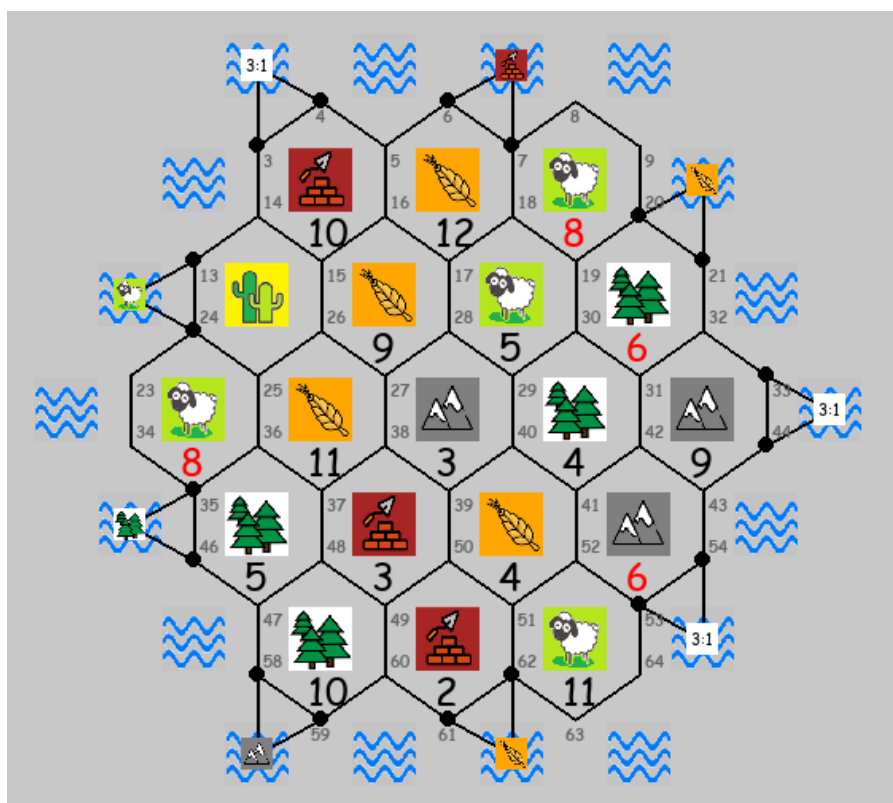
The Button class (`gui.py`) tracks button states for the interaction panel. These classes, detailed in Annex 2, are the backbone of the project’s initial state.

GUI Appearance

The GUI is a cornerstone of the first and second version, designed to visually represent the game’s state through three distinct parts: the playboard, scoreboard, and buttons for interaction.

The playboard, rendered by the GUI class using methods like `display_fresh_board()` and `_draw_tiles()`, displays the 45-tile board with hexagonal tiles, intersections, ports, and number chits.

Tiles are drawn with images (e.g., `Field.png`) from the Image enum, with red chits for high-probability values (6, 8) and black for others. All intersections are uniquely numbered (`intersection.id`). The first intersection on the playboard is numbered: 3. Dual ports are highlighted with black dots and show their type via a small image.



The scoreboard, managed by `update_scoreboard()`, displays player statistics (VP, C, S, R, LR, LA, RC, DC) with coloured player names, resource images (40×40), and development-card images (30×30). Vertical lines separate Statistics / Resource Cards / Development Cards.

	VP	C	S	R	R	A	E	LR	LA	RC	DC									
Player 1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
Player 2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
Player 3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
Player 4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				

VP = VictoryPoints

C = Number of Cities on playboard (max 4)

S = Number of Settlements on playboard (max 5)

R = Number of Settlements on playboard (max 15)

R = Extra VPs due to Longest Route (max 2)

A = Extra VPs due to Largest Army (max 2)

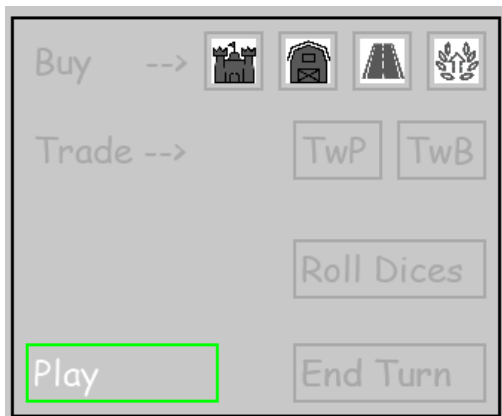
E = Extra VPs due to drawn VPs from Development Cards (max 5)

LR = Longest Route for specific player (example: 2)

LA = Largest Army for specific player (example: 0)

RC = Number of Resource Cards in players hand

DC = Number of Development Cards in players hand



The interaction panel managed by xxx shows:

- “Buy →”, “Trade →”, “Roll Dices”, “End Turn”, and “Play” buttons.

The panel is drawn with a black border. Button states are controlled by `set_button()` / `check_button()` and logged.

Added functionality since the first version:

- Dynamic guidance text (e.g., “Click a highlighted intersection for your settlement”).
- Pulsing coloured circles on every available settlement/road spot (via the central animation queue).
- OKY / OKN confirmation icons that appear exactly on the chosen location after a click.

Initial Placement

Initial Placement is guided by a flexible algorithm system implemented at the player level. The project now supports multiple placement strategies, giving both AI players and the human player (via simulation) access to different levels of sophistication.

Architecture

- **initial_placement.py** — Main orchestration module. It coordinates which algorithm to use based on settings and executes the placement logic.
- **algorithms_initial_placement.py** — Contains the actual implementation of all available algorithms.

Available Algorithms

1. **_max_pips** Places the settlement on the intersection with the highest total pip value (probability of resource production).
2. **_max_pips_and_ports** Extends the above by also considering proximity to favorable ports (especially 2:1 ports matching high-yield resources).
3. **_five_strategy_engine** A more advanced rule-based system using weighted combinations of multiple factors:
 - Balanced
 - Wood/Brick focused
 - Wheat/Ore focused
 - Wheat/Ore/Wool focused
 - Monopoly / High-probability focused
4. **Markov Chain Evaluator** The most sophisticated algorithm. It uses Markov Chain modeling to evaluate long-term expected value of each possible starting position.

Markov Evaluator Module

The markov_evaluator.py module was added as a separate file because it contains a significantly more complex implementation involving:

- Transition probability matrices
- Dynamic programming for efficient state value calculation
- Expected value computation across multiple future turns

This separation keeps the code clean and maintainable — the core logic remains isolated from the simpler heuristic algorithms.

The Markov approach was inspired by [Lauren Nagels' paper: "Analysis of 'The Settlers of Catan' Using Markov Chains"](#) (Texas Christian University, 2020s). We enhanced the original concept with dynamic programming techniques to make it computationally feasible for real-time use during initial placement.

Initial Placement is guided by an algorithm set on a player level. AI players use the original logic; the human player uses a fully interactive flow.

AI players (or when HUMAN_PLAYER=False):

1. Position settlement at intersection with highest pips.
2. Position road toward a convenient port (3:1 or 2:1 if ≥ 3 pips of that resource).
3. Immediate placement + GUI update.

Human player (when it is the human's turn):

1. Guidance starts in CHOOSING_SETTLEMENT – all legal intersections pulse.
2. Click a highlighted intersection → SETTLEMENT_SELECTED; OKY/OKN icons appear.
3. OKY → settlement placed (resources distributed if round == -1).
4. Automatically switches to CHOOSING_ROAD – connected roads pulse.
5. Click a road → ROAD_SELECTED + new OKY/OKN icons.
6. OKY → road placed.
7. “Play” button becomes active; clicking it advances to the next player.

Additional features (unchanged for both AI and human):

- Second settlement (round -1) triggers `distribute_initial_resources`.
- Road selection uses BFS (`_find_nearest_port`). [BFS = Breath-First Search](#).
- Every placement triggers GUI update with animation (settlement + road + optional resource-tile highlights).

Testing and Debugging

The `test_catan.py` module ensures the reliability of the initial state through unit and integration tests (board, game, player, GUI rendering). These tests (none been update after the first version) verify that the board is empty (no settlements, cities, or roads), players have zero resources, and the GUI renders correctly.

The main function mirrors `main.py`. Debugging is supported by extensive logging (`Board.write_debug_info()`, `Player.write_debug_info()`, `Game.write_debug_info()`, plus new guidance-state logging). All new human-placement flows have been manually verified with mouse-click sequences.

The project is now ready for the Execution phase (buying, trading, development cards) for both human and AI players.

Annex References

- Annex 1: Directory Structure – Detailed list of modules.
- Annex 2: Classes – Detailed list of classes.
 - Alphabetical
 - Classes per Module
- Annex 3: Attributes – Detailed list of attributes per class.
- Annex 4: Methods/Functions – Detailed list of methods/ functions and their usage.
 - Alphabetical
 - Methods/Functions per Class
 - Methods/Functions per Module
 - Methods/Functions per Module including description
- Annex 5: Example.

Annex 1: Directory Structure	Page: 7
Annex 2: Classes - Alphabetical	Page: 8
- Per Module	Page: 9
Annex 3: Attributes - Per Class	Page: 10 - Board
	Page: 11 - Game
	Page: 12 - Gui
	Page: 13 - Image (continued)
	Page: 14 - Player
	Page: 15 - Road
	Page: 17 - Tile
Annex 4: Methods/Functions - Alphabetical	Page: 17
- Per Class	Page: 19
- Per Module	Page: 21
- Per Module, incl description	Page: 23
Annex 5: Example - Board & Tiles	Page: 26
- Intersections	Page: 34
- Roads	Page: 37
- GUI	Page: 38

Annex 1: Directory Structure & modules

```
project/
├── docs/
│   └── Overview.pdf
├── core/
│   ├── __init__.py
│   ├── board.py
│   ├── player.py
│   ├── game.py
│   └── constants.py
├── gui/
│   ├── __init__.py
│   ├── gui.py
│   ├── gui_guidance.py
│   ├── gui_human_player.py
│   ├── gui_constants.py
│   └── event_handler.py
├── assets/
│   ├── images/
│   └── sounds/
├── test_catan.py
└── main.py
```

Annex 2: Classes

Alphabetical:

Class Name	Module	Description
Board	board.py	Manages the game board, including tiles, intersections, roads, and ports. Stores the board configuration (e.g., "Base_Random") and provides methods for rendering and updating board elements (e.g., placing settlements, roads).
Button	gui.py	Represents a GUI button with a name, display state, and switch status. Tracks button visibility and state (e.g., active/inactive) for rendering in the game interface.
Color	gui_constants.py	Enum defining color constants (e.g., `LGRAY`, `GREEN`, `GRAY`, `BLUE`) used for rendering GUI elements, buttons, and board tiles.
Corner	board.py	Represents a corner (vertex) of a hexagonal tile on the board. Tracks its location (e.g., 'N', 'EH'), structure type (e.g., 'Blank', 'Settlement', 'City'), player color, port type (if applicable), and associated intersection ID.
Edge	board.py	Represents an edge of a hexagonal tile on the board. Tracks its location (e.g., 'NE', 'E'), structure type (e.g., 'Blank', 'Road'), player color, and associated road ID as a tuple of two intersection IDs.
EventHandler	event_handler.py	Processes all mouse clicks and routes them correctly (first to confirmation icons, then to the "Play" button, then to HumanGuidance).
Font	gui_constants.py	Enum defining font sizes (e.g., `SMALL`, `NORMAL`, `LARGE`) and styles (e.g., `regular`, `bold`) for rendering text in the GUI.
Game	game.py	Manages core game logic, including game state (phase, state, turn), players, board, and development cards. Initializes the game with a sequence number, ID, and board configuration.
GUI	gui.py	Manages the game's graphical user interface. Handles button states, modes, board rendering, the central animation queue for pulsing highlights and quarter-circle reveal animations, updates the scoreboard and round/turn display, handles screenshots, and integrates the HumanGuidance system for interactive initial placement.
GUIHumanPlayer	gui_human_player.py	Manages rendering and dynamic active/inactive states of all interaction-panel buttons (Buy City, Buy Settlement, Buy Road, Buy DCard, TwP, TwB, Roll Dices, End Turn, Cancel, and the "Play" button) based on game phase and current HumanGuidance state.
HumanGuidance	gui_guidance.py	State controller regarding human initial placement (CHOOSING_SETTLEMENT → SETTLEMENT_SELECTED → CHOOSING_ROAD → ROAD_SELECTED → IDLE), pulsing highlights, and on-screen OKY/OKN confirmation icons.
Image	gui_constants.py	Enum defining image paths and sizes (e.g., `CITY_GREEN`, `CITY_DGRAY` with `(40, 40)`, `(30, 30)`) for buttons and board graphics. Manages the `IMAGES` dictionary for rendering.
Intersection	board.py	Represents a vertex on the board, tracking building status, port information, and tile associations.
PlacementState	gui_guidance.py	Enum defining the five placement states (IDLE, CHOOSING_SETTLEMENT, SETTLEMENT_SELECTED, CHOOSING_ROAD, ROAD_SELECTED).
Player	player.py	Represents a player, managing resources (e.g., `WHEAT`, `ORE`), development cards, settlements, cities, roads, and game statistics (e.g., victory points).
PlayerColor	gui_constants.py	Enum defining player color mappings (e.g., `BLUE`, `RED`, `WHITE`, `ORANGE`) with codes and RGB values for game logic and rendering.
ResourceCard	constants.py	Enum defining resource types (e.g., `WHEAT`, `ORE`, `WOOL`) used for player resource management and development card purchases.
ResourceCardDashboard	game.py	Tracks resource card distribution across the game, including total resources and per-player views.
Road	board.py	Represents an edge on the board, tracking occupancy status and player color.
Settings	game.py	Manages game settings, including human player status, strategy weights, and port text.
Sound	gui_constants.py	Enum defining sound effect paths (e.g., `DICEROLL`, `BUTTON`) for GUI interactions.
StrategyDashboard	game.py	Tracks player statistics (e.g., victory points, settlements) for the scoreboard.
Tile	board.py	Represents a hexagonal tile with type (e.g., `Field`, `Sea`), value, and color.
TurnDetails	game.py	Tracks per-turn details (e.g., dice rolls, trades, built structures) for game progression.

Classes per module:

Class Name	Module	Description
Board	board.py	Manages the game board, including tiles, intersections, roads, and ports. Stores the board configuration (e.g., "Base_Random") and provides methods for rendering and updating board elements (e.g., placing settlements, roads).
Corner	board.py	Represents a corner (vertex) of a hexagonal tile on the board. Tracks its location (e.g., 'N', 'EH'), structure type (e.g., 'Blank', 'Settlement', 'City'), player color, port type (if applicable), and associated intersection ID.
Edge	board.py	Represents an edge of a hexagonal tile on the board. Tracks its location (e.g., 'NE', 'E'), structure type (e.g., 'Blank', 'Road'), player color, and associated road ID as a tuple of two intersection IDs.
Intersection	board.py	Represents a vertex on the board, tracking building status, port information, and tile associations.
Road	board.py	Represents an edge on the board, tracking occupancy status and player color.
Tile	board.py	Represents a hexagonal tile with type (e.g., 'Field', 'Sea'), value, and color.
Player	player.py	Represents a player, managing resources (e.g., 'WHEAT', 'ORE'), development cards, settlements, cities, roads, and game statistics (e.g., victory points).
Game	game.py	Manages core game logic, including game state (phase, state, turn), players, board, and development cards. Initializes the game with a sequence number, ID, and board configuration.
ResourceCardDashboard	game.py	Tracks resource card distribution across the game, including total resources and per-player views.
StrategyDashboard	game.py	Tracks player statistics (e.g., victory points, settlements) for the scoreboard.
TurnDetails	game.py	Tracks per-turn details (e.g., dice rolls, trades, built structures) for game progression.
Settings	game.py	Manages game settings, including human player status, strategy weights, and port text.
ResourceCard	constants.py	Enum defining resource types (e.g., 'WHEAT', 'ORE', 'WOOL') used for player resource management and development card purchases.
Button	gui.py	Represents a GUI button with a name, display state, and switch status. Tracks button visibility and state (e.g., active/inactive) for rendering in the game interface.
GUI	gui.py	Manages the game's graphical user interface. Handles button states, modes, board rendering, the central animation queue for pulsing highlights and quarter-circle reveal animations, updates the scoreboard and round/turn display, handles screenshots, and integrates the HumanGuidance system for interactive initial placement.
GUIHumanPlayer	gui_human_player.py	Manages rendering and dynamic active/inactive states of all interaction-panel buttons (Buy City, Buy Settlement, Buy Road, Buy DCard, TwP, TwB, Roll Dices, End Turn, Cancel, and the "Play" button) based on game phase and current HumanGuidance state.
Color	gui_constants.py	Enum defining color constants (e.g., 'LGRAY', 'GREEN', 'GRAY', 'BLUE') used for rendering GUI elements, buttons, and board tiles.
Font	gui_constants.py	Enum defining font sizes (e.g., 'SMALL', 'NORMAL', 'LARGE') and styles (e.g., 'regular', 'bold') for rendering text in the GUI.
Image	gui_constants.py	Enum defining image paths and sizes (e.g., 'CITY_GREEN', 'CITY_DGRAY' with '(40, 40)', '(30, 30)') for buttons and board graphics. Manages the 'IMAGES' dictionary for rendering.
PlayerColor	gui_constants.py	Enum defining player color mappings (e.g., 'BLUE', 'RED', 'WHITE', 'ORANGE') with codes and RGB values for game logic and rendering.
Sound	gui_constants.py	Enum defining sound effect paths (e.g., 'DICEROLL', 'BUTTON') for GUI interactions.
EventHandler	gui_guidance.py	Processes all mouse clicks and routes them correctly (first to confirmation icons, then to the "Play" button, then to HumanGuidance).
HumanGuidance	gui_guidance.py	State controller regarding human initial placement (CHOOSING_SETTLEMENT → SETTLEMENT_SELECTED → CHOOSING_ROAD → ROAD_SELECTED → IDLE), pulsing highlights, and on-screen OKY/OKN confirmation icons.
PlacementState	gui_guidance.py	Enum defining the five placement states (IDLE, CHOOSING_SETTLEMENT, SETTLEMENT_SELECTED, CHOOSING_ROAD, ROAD_SELECTED).

Annex 3: Attributes per Class

Attribute Name	Type	Brief Description	Used
Class Board			
LIST_OF_LAND_TILES	List[int]	List of land tile ids (e.g. [9, 10, 11, ...]).	Yes
LIST_OF_SKIPPED_TILE_IDS	List[int]	List of tile ids (e.g. [0, 1, 6, 7, ...]) in BOARD_LAYOUT not shown in GUI.	Yes
INTERSECTION_IN_WATER	List[int]	List of intersection ids in water (e.g. [0, 1, 2]).	Yes
INTERSECTIONS_ARE_PORT	List[List[int]]	List of intersection ID pairs that have ports (e.g., [[3, 4], [6, 7], ...]).	Yes
BOARD_LAYOUT	List[List[int]]	2D list defining tile positions (e.g. [[0, 0, 2, ...]]).	Yes
LIST_OF_PORTTYPES	List[str]	List of port types (["3:1", "3:1", ..., "2:1 Wool"]) used in _add_ports().	Yes
ALL_TILE_IDS	Set[int]	Set of all tile IDs (0–45).	Yes
board_name	str	Name of the board configuration (e.g. "Base_Random").	Yes
round	int	Current game round number.	Yes
turn	int	Current game turn number.	Yes
intersections	List[Intersection]	List of 67 intersection objects.	Yes
roads	List[Road]	List of road objects, dynamically populated based on board edges.	Yes
list_of_roads_connected_to_intersection	List[List[Tuple[int, int]]]	List of roads connected to each intersection, populated in _create_list_of_roads_connected_to_intersection().	Yes
tiles	List[Optional[Tile]]	List of tile objects for all tile ids.	Yes
Class Button			
name	str	Name of the button (e.g. "buy_city").	Yes
display_tf	bool	Whether the button is displayed (True/ False).	Yes
switched_tf	bool	Whether the button's display state has changed (True/ False).	Yes
Class Color			
WHITE	Tuple[int, int, int]	RGB color (255, 255, 255)	Yes
BLACK	Tuple[int, int, int]	RGB color (0, 0, 0)	Yes
LGRAY	Tuple[int, int, int]	RGB color (200, 200, 200)	Yes
DGRAY	Tuple[int, int, int]	RGB color (100, 100, 100)	Yes
GRAY	Tuple[int, int, int]	RGB color (169, 169, 169)	Yes
GREEN	Tuple[int, int, int]	RGB color (0, 255, 0)	Yes
BLUE	Tuple[int, int, int]	RGB color (0, 0, 255)	Yes
RED	Tuple[int, int, int]	RGB color (255, 0, 0)	Yes
ORANGE	Tuple[int, int, int]	RGB color (255, 165, 0)	Yes
FIELD	Tuple[int, int, int]	RGB color (255, 255, 153)	Yes
MOUNTAIN	Tuple[int, int, int]	RGB color (139, 69, 19)	Yes
WOODS	Tuple[int, int, int]	RGB color (0, 100, 0)	Yes
HILLS	Tuple[int, int, int]	RGB color (204, 0, 0)	Yes
GRASS	Tuple[int, int, int]	RGB color (173, 255, 47)	Yes
DESERT	Tuple[int, int, int]	RGB color (245, 245, 220)	Yes
SEA	Tuple[int, int, int]	RGB color (0, 191, 255)	Yes
Class EventHandler			
(none)		Stateless handler	
Class Font			
SMALL	Tuple[str, int]	Font definition ("Comic Sans MS", 10)	Yes
NORMAL	Tuple[str, int]	Font definition ("Comic Sans MS", 16)	Yes
LARGE	Tuple[str, int]	Font definition ("Comic Sans MS", 24)	Yes
Class Game			
manager	None	Placeholder for game manager.	No
sequence_number	int	Game sequence number (e.g. 1).	No
id	str	Unique game id (e.g. "20250815").	Yes
time_ended	Optional[str]	Time the game ended (None initially).	No
phase	str	Game phase (e.g. "Initial Placement").	No
state	str	Game state (e.g. "None").	No
state_1	str	Additional state info (e.g. "0").	No
state_2	str	Additional state info (e.g. "0").	No
round	int	Current round number (e.g. -2 initially).	Yes
turn	int	Current turn number (e.g. 1 initially)	Yes
players	List[Player]	List of player objects.	Yes
board	Board	Game board instance.	Yes
gui	Optional[GUI]	GUI instance (None initially).	Yes
ip	None	Placeholder for InitialPlacement.	Yes
dice_roll	Optional[Tuple[int, int]]	Current dice roll (None initially).	No
dice_rolls	List[Tuple[int, int]]	History of dice rolls.	No
dice_roll_history	List[int]	Frequency of dice roll sums (indices 0–12).	No
dice_roll_matrix	List[str]	Placeholder for dice roll matrix (empty).	No
dcards_stack	List	Stack of shuffled development card types (e.g., ['knight', 'victory_point', ...]).	No
robber_tile_probabilities	List[List[Union[int, float]]]	Probabilities for robber placement on tiles.	No
previous_tile_having_robber	List[int]	Last tiles with the robber (e.g. [0, 0, 0]).	No
list_of_tiles_having_robber	List	History of tiles with the robber (empty).	No
last_total_turn_with_dr7	int	Last turn with a dice roll of 7 (0 initially).	No
settings_tf	bool	Settings enabled (True/ False).	No
settings	Settings	Game settings instance.	No

Attribute Name	Type	Brief Description	Used
Class Game (continued)			
initial_placement_balanced	List	Balanced strategy data (empty).	No
initial_placement_wood_brick	List	Wood/ brick strategy data (empty).	No
initial_placement_wheat_ore	List	Wheat/ ore strategy data (empty).	No
initial_placement_wheat_ore_wool	List	Wheat/ ore/ wool strategy data (empty).	No
initial_placement_monopoly	List	Monopoly strategy data (empty).	No
resource_production_probability	List[List[int]]	Resource production probabilities per player.	No
tile_type	List	Tile type data (empty).	No
resource_type_available	List	List of available trade-with types.	No
resource_type_occupied	List	List of occupied trade-with types.	No
resource_type_players	List	List of resource types per player.	No
players_impacted	List[bool]	Players impacted by actions (4x False initially).	No
common_next_settlements	List	Common next settlement locations (empty).	No
common_new_settlements	List	Common new settlement locations (empty).	No
common_next_roads	List	Common next road locations (empty).	No
last_known_strategies	List[List[Union[List[int], int]]]	Last known player strategies.	No
last_known_outlooks	List[List[Union[str, List, int]]]	Last known player outlooks.	No
current_player	Optional[Player]	Current player (None initially).	No
winner	Optional[Player]	Game winner (None initially).	No
game_over	bool	Game over status (False initially).	No
longest_route_player	Optional[Player]	Player with longest route (None initially).	No
largest_army_player	Optional[Player]	Player with largest army (None initially).	No
strategy_dashboard	List[StrategyDashboard]	List of strategy dashboards for players.	Yes
resource_card_dashboard	List[ResourceCardDashboard]	Resource card distribution data.	No
myturn	TurnDetails	Turn details instance for current turn.	No
Class GUI			
round	int	Current game round number.	Yes
turn	int	Current game turn number.	Yes
buttons	List[Button]	List of button objects for GUI interactions.	Yes
modes	List	List of mode objects (stub, for future use).	Yes
human_guidance	HumanGuidance	Instance of the human placement state controller	Yes
animate_queue_elements	List[Tuple[...]]	Central animation queue for pulsing highlights and quarter-circle reveals	Yes
Class HumanGuidance			
state	PlacementState	Current placement state.	Yes
player	Optional[Player]	Human player currently placing	Yes
placement_step	Optional[int]	Current step in initial placement sequence	Yes
selected_inter_id	Optional[int]	Selected intersection (settlement)	Yes
selected_road	Optional[Tuple[int, int]]	Selected road after settlement	Yes
available_inters	List[int]	Valid settlement spots (pulsing highlights)	Yes
available_roads	List[Tuple[int, int]]	Valid roads from selected settlement	Yes
confirm_center	Optional[Tuple[int, int]]	Screen position for OKY/OKN icons	Yes
last_pulse	int	Timestamp of last pulsing animation	Yes
Class GUIHumanPlayer			
(none)	-	No attributes defined in __init__.	
Class Image			
DICE_1	Tuple[str, Tuple[int, int]]	Image path and size for dice 1 ("1.png", (75, 75)).	No
DICE_2	Tuple[str, Tuple[int, int]]	Image path and size for dice 2 ("2.png", (75, 75)).	No
DICE_2B	Tuple[str, Tuple[int, int]]	Image path and size for dice 2b ("2b.png", (75, 75)).	No
DICE_3	Tuple[str, Tuple[int, int]]	Image path and size for dice 3 ("3.png", (75, 75)).	No
DICE_3B	Tuple[str, Tuple[int, int]]	Image path and size for dice 3b ("3b.png", (75, 75)).	No
DICE_4	Tuple[str, Tuple[int, int]]	Image path and size for dice 4 ("4.png", (75, 75)).	No
DICE_5	Tuple[str, Tuple[int, int]]	Image path and size for dice 5 ("5.png", (75, 75)).	No
DICE_6	Tuple[str, Tuple[int, int]]	Image path and size for dice 6 ("6.png", (75, 75)).	No
DICE_6B	Tuple[str, Tuple[int, int]]	Image path and size for dice 6b ("6b.png", (75, 75)).	No
MOUNTAIN	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Mountain ("Mountain3.png", [(40, 40), (20, 20)]).	Yes
FIELD	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Field ("Field2.png", [(40, 40), (20, 20)]).	Yes
WOODS	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Woods ("Woods.png", [(40, 40), (20, 20)]).	Yes
HILLS	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Hills ("Hills3.png", [(40, 40), (20, 20)]).	Yes
GRASS	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Grass ("Grass.png", [(40, 40), (20, 20)]).	Yes
DESERT	Tuple[str, List[Tuple[int, int]]]	Image path and size for Desert ("Desert.png", (40, 40)).	Yes
SEA	Tuple[str, List[Tuple[int, int]]]	Image path and size for Sea ("Sea3.png", (40, 40)).	Yes
PLUS	Tuple[str, List[Tuple[int, int]]]	Image path and size for Plus ("Plus.png", (50, 50)).	No
MIN	Tuple[str, List[Tuple[int, int]]]	Image path and size for Min ("Min.png", (50, 50)).	No
FINISH	Tuple[str, List[Tuple[int, int]]]	Image path and size for Finish ("Finish2.png", (50, 50)).	No
ROBBER	Tuple[str, List[Tuple[int, int]]]	Image path and size for Robber ("Robber.png", (40, 40)).	No
SETTINGS_ON	Tuple[str, List[Tuple[int, int]]]	Image path and size for Settings On ("Settings3.png", (40, 40)).	No
SETTINGS_OFF	Tuple[str, List[Tuple[int, int]]]	Image path and size for Settings Off ("Settings.png", (40, 40)).	No

Attribute Name	Type	Brief Description	Used
Class Image (continued)			
QUESTIONMARK	Tuple[str, List[Tuple[int, int]]]	Image path and size for Questionmark ("Questionmark.png", (40, 40)).	No
DC_VPOINT	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Victory Point card ("vp.jpg", [(20, 20), (30, 30), (40, 40)]).	Yes
DC_KNIGHT	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Knight card ("knight.png", [(20, 20), (30, 30), (40, 40)]).	Yes
DC_ROAD	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Road card ("road.jpg", [(20, 20), (30, 30), (40, 40)]).	Yes
DC_PLENTY	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Plenty card ("plenty2.png", [(20, 20), (30, 30), (40, 40)]).	Yes
DC_MONOPOLY	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Monopoly card ("monopoly.png", [(20, 20), (30, 30), (40, 40)]).	Yes
OKY	Tuple[str, List[Tuple[int, int]]]	Image path and size for OKY ("OK.png", (40, 40)).	No
OKN	Tuple[str, List[Tuple[int, int]]]	Image path and size for OKN ("OK_pale.png", (40, 40)).	No
NOK	Tuple[str, List[Tuple[int, int]]]	Image path and size for NOK ("NOK.png", (40, 40)).	No
CITY_BLUE	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Blue City ("CityBlue4s.png", [(30, 30), (40, 40)]).	No
SETTLEMENT_BLUE	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Blue Settlement ("BarnBlue5s.png", [(30, 30), (40, 40)]).	No
CITY_RED	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Red City ("CityRed4s.png", [(30, 30), (40, 40)]).	No
SETTLEMENT_RED	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Red Settlement ("BarnRed4s.png", [(30, 30), (40, 40)]).	No
CITY_WHITE	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for White City ("CityWhite3s.png", [(30, 30), (40, 40)]).	No
SETTLEMENT_WHITE	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for White Settlement ("BarnWhite2s.png", [(30, 30), (40, 40)]).	No
CITY_ORANGE	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Orange City ("CityOrange4s.png", [(30, 30), (40, 40)]).	No
SETTLEMENT_ORANGE	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Orange Settlement ("BarnOrange4s.png", [(30, 30), (40, 40)]).	No
CITY_GREEN	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Green City ("CityGreen1s.png", [(30, 30), (40, 40)]).	No
SETTLEMENT_GREEN	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Green Settlement ("BarnGreen2s.png", [(30, 30), (40, 40)]).	No
ROAD_GREEN	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Green Road ("RoadGreen.png", [(30, 30), (40, 40)]).	No
DCARD_GREEN	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Green DCard ("DCardGreen.png", [(30, 30), (40, 40)]).	No
TWP_GREEN	Tuple[str, List[Tuple[int, int]]]	Image path and size for Green TwP ("TwPGreen.png", (40, 40)).	No
CITY_DGRAY	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Dark Gray City ("CityDGray1s.png", [(30, 30), (40, 40)]).	Yes
SETTLEMENT_DGRAY	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Dark Gray Settlement ("BarnDGray2s.png", [(30, 30), (40, 40)]).	Yes
ROAD_DGRAY	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Dark Gray Road ("RoadGray.png", [(30, 30), (40, 40)]).	Yes
DCARD_DGRAY	Tuple[str, List[Tuple[int, int]]]	Image path and sizes for Dark Gray DCard ("DCardGray.png", [(30, 30), (40, 40)]).	Yes
TWP_RED	Tuple[str, List[Tuple[int, int]]]	Image path and size for Red TwP ("TwPRed.png", (40, 40)).	Yes
Class Intersection			
id	int	Unique intersection id (0–66).	Yes
face	str	Type of structure built on the intersection ('Blank', 'Settlement', 'City').	Yes
occupied_tf	bool	Occupancy status (True/ False).	Yes
color	str	Player color or "Blank".	Yes
port_tf	bool	Port presence (True/ False).	Yes
port_type	str	Port type (e.g. "3:1", "2:1 Wheat").	Yes
three_tile_ids	List[int]	List of three adjacent tile ids.	Yes
three_tile_types	List[str]	List of types of the three adjacent tiles.	Yes
three_tile_values	List[int]	List of values of the three adjacent tiles.	Yes
three_tile_probabilities	List[float]	Probabilities for adjacent tiles.	Yes
all_tile_types	List[int]	Counts of resource types for adjacent tiles ([Wheat, Ore, Wood, Brick, Wool]).	Yes
all_tile_values	List[int]	Counts of tile values (2–12) for adjacent tiles.	Yes
all_tile_probabilities	List[float]	Sum of probabilities for each resource type.	Yes
three_roads	List[Tuple[int, int]]	List of roads connected to the intersection.	Yes
three_intersection_ids	List[int]	List of adjacent intersection IDs	Yes
Class Player			
game	None	Placeholder for game reference.	Yes
color	str	Player color (e.g. "Blue").	Yes
color2	str	Secondary color (same as color initially).	Yes
id	int	Unique player id (e.g. 2 for human player).	Yes
gameover_tf	bool	Game over status (True/ False).	Yes
sequence	int	Turn order (1–4).	Yes
points	int	Total points (0 initially).	Yes
longest_route_tf	bool	Whether player has longest road (True/ False).	Yes
size_longest_road	int	Length of longest road (0 initially).	Yes
structure_longest_road	List[Tuple[int, int]]	Roads in longest road (empty).	Yes
number_of_clusters	int	Number of road clusters (0 initially).	Yes
structure_of_clusters	List	Structure of road clusters (empty).	Yes
largest_army_tf	bool	Whether player has largest army (True/ False).	Yes
size_largest_army	int	Number of knights played (0 initially).	Yes
number_of_rcards	int	Number of resource cards (0 initially).	Yes
number_of_dcards	int	Number of development cards (0 initially).	Yes
rcards	Dict[ResourceCard, int]	Resource card counts (e.g. [WHEAT: 0]).	Yes
dcard_summary	List[List[Union[str, int]]]	Summary of development cards (e.g. [{"victory_point": 0, 0, 0}]).	No
development_cards	List[str]	List of development cards (empty).	No
victory_points	int	Victory points (0 initially).	Yes
settlements	List[int]	List of settlement intersection ids (empty).	Yes
cities	List[int]	List of city intersection ids (empty).	Yes
roads	List[Tuple[int, int]]	List of road ids (empty).	Yes

Attribute Name	Type	Brief Description	Used
Class Player (continued)			
turn_details_RP	List[int]	Resource production details (e.g. [0, 0, 0, 0, 0, 0]).	No
turn_details_RP_robber	List[int]	Robber-related details (e.g. [0, 0, 0, 0, 0, 0]).	No
turn_details_buy	List[int]	Purchase details (e.g. [0, 0, 0, 0, 0, 0]).	No
turn_details_steal	List[int]	Steal details (e.g. [0, 0, 0, 0, 0, 0]).	No
turn_details_discard	List[int]	Discard details (e.g. [0, 0, 0, 0, 0, 0]).	No
turn_details_TwP	List[int]	Trade-with-player details (e.g. [0, 0, 0, 0, 0, 0]).	No
turn_details_last_TwPdeal	List[int]	Last trade-with-player deal (e.g. [0, 0, 0, 0, 0, 0]).	No
turn_details_TwB	List[int]	Trade-with-bank details (e.g. [0, 0, 0, 0, 0, 0]).	No
turn_details_dcard	List[int]	Development card details (e.g. [0, 0, 0, 0, 0, 0]).	No
trade_rates	Dict[ResourceCard, int]	Trade rates for resources (e.g. {WHEAT: 4}).	No
port_access	Dict[str, bool]	Port access status (e.g. {"3:1": False}).	No
last_action	str	Last action taken (e.g. "None").	No
Class PlayerColor			
BLUE	Tuple[int, str, Tuple[int, int, int]]	Player color (1, "Blue", (0, 0, 255)).	Yes
RED	Tuple[int, str, Tuple[int, int, int]]	Player color (2, "Red", (255, 0, 0)).	Yes
WHITE	Tuple[int, str, Tuple[int, int, int]]	Player color (3, "White", (255, 255, 255)).	Yes
ORANGE	Tuple[int, str, Tuple[int, int, int]]	Player color (4, "Orange", (255, 165, 0)).	Yes
code	int	Player color code (e.g. "Blue" -> 1 for BLUE).	Yes
color_name	str	Color name (e.g. "Blue").	Yes
rgb	Tuple[int, int, int]	RGB values for the color.	Yes
Class ResourceCard			
WHEAT	str	Resource type for wheat ("Wheat").	Yes
ORE	str	Resource type for ore ("Ore").	Yes
WOOD	str	Resource type for wood ("Wood").	Yes
BRICK	str	Resource type for brick ("Brick").	Yes
WOOL	str	Resource type for wool ("Wool").	Yes
Class ResourceCardDashboard			
resource_production_game_total	List[int]	Total resources distributed (e.g. [0, 0, 0, 0, 0, 0]).	No
resource_production_game_player	List[List[int]]	Per-player resources (e.g., [[1, 0, 0, 0, 0, 0], ...]).	No
resource_production_game_player_view	List[List[int]]	Players' views of others' resources.	No
Class Road			
id	Tuple[int, int]	Tuple of two intersection ids defining the road.	Yes
occupied_tf	bool	Whether the road is occupied (True/ False)".	Yes
kind	str	Road type ("Road" or "Blank").	Yes
color	str	Player color or "Blank".	Yes
two_tiles	List[List[any]]	List of adjacent tiles with their attributes ([tile_id, type, value]).	Yes
Class Settings			
human_player_tf	bool	Whether human player is enabled (True/ False).	No
human_player_sequence	int	Human player sequence (e.g. 3).	No
topx_tf	bool	Whether top-x is enabled (True/ False).	No
topx	int	Top-x value (e.g. 15).	No
weight_balanced	float	Weight for balanced strategy (e.g. 1.0).	No
weight_wood_brick	float	Weight for wood/ brick strategy (e.g. 1.0).	No
weight_wheat_ore	float	Weight for wheat/ ore strategy (e.g. 1.0).	No
weight_wheat_ore_wool	float	Weight for wheat/ ore/ wool strategy (e.g. 0.15).	No
weight_monopoly	float	Weight for monopoly strategy (e.g. 1.0).	No
weight_probability	float	Weight for probability-based strategy (e.g. 1.0).	No
weight_blocked	float	Weight for blocked strategy (e.g. 0,2).	No
user_text1	str	Harbor user text 1 (e.g. "3").	No
user_text2	str	Harbor user text 2 (e.g. "2").	No
user_text3	str	Harbor user text 3 (e.g. "1").	No
Class Sound			
DICEROLL	str	Path to dice roll sound ("DiceRoll.wav").	No
BUTTON	str	Path to button click sound ("button-click-3.wav").	No
BUTTONHP	str	Path to human player button sound ("Bell2.wav").	No
BUILDRoad	str	Path to build road sound ("BuildRoad.wav").	No
FANFARE	str	Path to fanfare sound ("fanfare-2.wav").	No
BELL	str	Path to success bell sound ("success-bell.wav").	No
ERROR	str	Path to error sound ("Error-sound.wav").	No
DANGER	str	Path to danger sound ("Danger.wav").	No
STEAL	str	Path to steal sound ("CashRegister.wav").	No
DEAL	str	Path to deal sound ("CashRegister.wav").	No
NOTWPFFOUND	str	Path to no TwP found sound ("No_TwP_Found.wav").	No
TWPFFOUND	str	Path to TwP found sound ("TwP_Found.wav").	No
TWPFFOUND2	str	Path to TwP found alternate sound ("infobleep.wav").	No
BUYDCARD	str	Path to buy DCard sound ("BuyDCard2.wav").	No
PLAYDCARD	str	Path to play DCard sound ("PlayDCard.wav").	No
NEXTTURN	str	Path to next turn sound ("NoGui_NextTurn.wav").	No
NEXTGAME	str	Path to next game sound ("NoGui_NextGame.wav").	No
MIDGAME	str	Path to mid-game sound ("NoGui_Midgame.wav").	No
ENDGAME	str	Path to end game sound ("NoGui_Endgame.wav").	No

Attribute Name	Type	Brief Description	Used
Class StrategyDashboard			
player_id	int	Player id (1–4).	Yes
victory_points	int	Total victory points (0 initially).	Yes
number_of_settlements	int	Number of settlements (0 initially).	Yes
number_of_cities	int	Number of cities (0 initially).	Yes
victory_points_dcard	int	Victory points from development cards (0 initially).	Yes
longest_route	int	Length of longest route (0 initially).	Yes
largest_army	int	Number of knights played (0 initially).	Yes
number_of_rcards	int	Number of resource cards (0 initially).	Yes
number_of_dcards	int	Number of development cards (0 initially).	Yes
distribution_of_tile_values	str	Distribution of tile values (e.g. "00000X00000").	No
distribution_of_tile_types	str	Distribution of tile types (e.g. "000000").	No
Class Tile			
id	int	Unique tile id (e.g. 9 for a land tile).	Yes
type	str	Resource type (e.g. "Field", "Desert", "Sea").	Yes
value	int	Tile number (2–12 for land 0 for Desert/Sea).	Yes
color	str	Tile color (e.g. "Blank", "Sea").	Yes
edges	List[dict]	List of edge dictionaries (representing Edge objects).	Yes
corners	List[dict]	List of corner dictionaries (representing Corner objects).	Yes
occupied_tf	bool	Occupancy status (True/ False).	Yes
Class TurnDetails			
round	int	Current game round number.	No
turn	int	Current game turn number.	No
dice_roll	int	Sum of the dice roll (0 initially).	No
validate_function_enough	bool	Whether enough resources are available.	No
validate_function_TwP_Match	bool	Whether trade with player matches.	No
validate_function_discard_rcards_by_HP	bool	Whether human player must discard cards.	No
validate_function_set_robber_by_HP	bool	Whether human player must set robber.	No
validate_function_outlook_opponents_for_H	bool	Whether to outlook opponents for human player.	No
validate_function_built_two_roads	int	Number of roads built this turn (0 initially).	No
road_built_in_turn_tf	bool	Whether a road was built this turn (True/ False).	No
roads_built_in_turn	List[Tuple[int, int]]	Roads built this turn (empty).	No
settlement_built_in_turn_tf	bool	Whether a settlement was built this turn (True/ False).	No
settlements_built_in_turn	List[int]	Settlements built this turn (empty).	No
city_built_in_turn_tf	bool	Whether a city was built this turn (True/ False).	No
cities_built_in_turn	List[int]	Cities built this turn (empty).	No
question_mark_button	List[int]	Status of question mark buttons per player.	No
dcard_played_in_turn	List[int]	Development cards played this turn (e.g. [0, 0, 0, 0, 0]).	No
dcard_played_in_turn_tf	bool	Whether a development card was played (True/ False).	No
tile_type_selected_1	List[int]	First selected tile types (e.g. [0, 0, 0, 0, 0]).	No
tile_type_selected_2	List[int]	Second selected tile types (e.g. [0, 0, 0, 0, 0]).	No
players_having_too_many_rcards	List[int]	Players with too many resource cards.	No
rcard_give	List[int]	Resource cards given in trades (e.g. [0, 0, 0, 0, 0]).	No
rcard_get	List[int]	Resource cards received in trades (e.g. [0, 0, 0, 0, 0]).	No
list_of_TwP	List	List of trade-with-player deals (empty).	No
number_of_deals_offered	int	Number of trade deals offered (0 initially).	No
list_of_TwP_rejected_by_HP	List	Trade deals rejected by human player (empty).	No
list_of_TwHP	List[int]	Trade-with-human-player data (e.g. [0, 0, 0, 0, 0]).	No
dcard_selected	List[int]	Selected development cards (e.g. [0, 0, 0, 0, 0]).	No
modes	List	List of active modes (empty).	No

Annex 4: Methods/Functions

Alphabetical:

Function/ Method	Module	Class	Used
Board.__init__	board.py	Board	Yes
Board.add_ports	board.py	Board	Yes
Board.add_intersections	board.py	Board	Yes
Board.add_roads	board.py	Board	Yes
Board.add_tiles	board.py	Board	Yes
Board.get_board	board.py	Board	Yes
Board.initialize_board	board.py	Board	Yes
Board.can_build_road_color	board.py	Board	No
Board.save_game	board.py	Board	Yes
Board.load_game	board.py	Board	Yes
Board.can_build_intersection	board.py	Board	No
Board.occupy_intersection	board.py	Board	No
Board.occupy_road	board.py	Board	No
Board.write_debug_info	board.py	Board	No
Button.__init__	gui.py	Button	Yes
cleanup_pygame	test_catan.py	None	No
convert_tile	gui.py	None	Yes
EventHandler.handle_click()	event_handler.py	EventHandler	Yes
Font.initialize_fonts	gui_constants.py	Font	Yes
Game.__init__	game.py	Game	Yes
Game.initialize_players	game.py	Game	Yes
Game.advance_turn	game.py	Game	No
Game.distribute_rcards	game.py	Game	No
Game.handle_okn_click	game.py	Game	No
Game.handle_oky_click	game.py	Game	No
Game.roll_dice	game.py	Game	No
Game.sync_round_turn	game.py	Game	No
Game.write_debug_info	game.py	Game	No
get_rcards	constants.py	None	Yes
GUI.__init__	gui.py	GUI	Yes
GUI.animate_element	gui.py	GUI	No
GUI.block_adjacent_in_gui()	gui.py	GUI	Yes
GUI.draw_hexagon_lines	gui.py	GUI	Yes
GUI.draw_intersections	gui.py	GUI	Yes
GUI.draw_ports	gui.py	GUI	Yes
GUI.draw_tile_values	gui.py	GUI	Yes
GUI.draw_tiles	gui.py	GUI	Yes
GUI.animate_continuous()	gui.py	GUI	Yes
GUI.occupy_city_in_gui()	gui.py	GUI	Yes
GUI.occupy_settlement_in_gui()	gui.py	GUI	Yes
GUI.occupy_road_in_gui()	gui.py	GUI	Yes
GUI.render_scoreboard_row()	gui.py	GUI	No
GUI.check_button	gui.py	GUI	Yes
GUI.check_mode	gui.py	GUI	Yes
GUI.display_fresh_board	gui.py	GUI	Yes
GUI.display_scoreboard	gui.py	GUI	Yes
GUI.draw_board_base()	gui.py	GUI	Yes
GUI.draw_all_permanent_buildings()	gui.py	GUI	Yes
GUI.draw_guidance()	gui.py	GUI	Yes
GUI.draw_guidance_text()	gui.py	GUI	Yes
GUI.print_queues()	gui.py	GUI	Yes
GUI.queue_latest_placement()	gui.py	GUI	No
GUI.handle_confirmation_click()	gui.py	GUI	Yes
GUI.set_button	gui.py	GUI	Yes
GUI.set_mode_duo	gui.py	GUI	No
GUI.update_board	gui.py	GUI	Yes
GUI.update_scoreboard()	gui.py	GUI	Yes

Function/ Method	Module	Class	Used
GUIHumanGuidance.start_settlement_phase()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.start_road_phase()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._highlight_available()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.clear()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.draw()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._compute_available_settlements()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._compute_available_roads()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._show_selected()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._find_clicked_inter()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._find_clicked_road()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._place_settlement()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._place_road()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._finish_human_turn()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.on_board_click()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.on_confirmation()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.is_placing()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanPlayer.__init__	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_analysis	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_city	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_dcard	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_road	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_settlement	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_cancel	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_end_turn	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_new_game	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_quit	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_roll_dices	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_tw_b	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_tw_p	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.check_modes1	gui_human_player.py	None	Yes
GUIHumanPlayer.check_modes2	gui_human_player.py	None	Yes
GUIHumanPlayer.get_hp_sequence_hp	gui_human_player.py	None	Yes
GUIHumanPlayer.log_spec2	gui_human_player.py	None	Yes
GUIHumanPlayer.show_buttons_HP	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.text_buy	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.text_trade	gui_human_player.py	GUIHumanPlayer	Yes
Image.initialize_images	gui_constants.py	Image	Yes
intersection_probability	constants.py	None	No
Intersection.__init__	board.py	Intersection	Yes
main	main.py	None	Yes
Player.__init__	player.py	Player	Yes
Player.add_rcard	player.py	Player	No
Player.build_structure	player.py	Player	No
Player.can_afford	player.py	Player	No
Player.get_resource_production_probability	player.py	Player	No
Player.rcards_in_hand	player.py	Player	Yes
Player.update_trade_rates	player.py	Player	No
Player.write_debug_info	player.py	Player	No
PlayerColor.__init__	gui_constants.py	PlayerColor	Yes
ResourceCardDashboard.__init__	game.py	ResourceCardDashboard	Yes
Road.__init__	board.py	Road	Yes
Settings.__init__	game.py	Settings	Yes
setup_pygame	test_catan.py	None	No
Sound.initialize_sounds	gui_constants.py	Sound	Yes
StrategyDashboard.__init__	game.py	StrategyDashboard	Yes
test_board_rendering	test_catan.py	None	No
test_button_display	test_catan.py	None	No
test_game_initialization	test_catan.py	None	No
test_gui_rendering	test_catan.py	None	No
test_player_initialization	test_catan.py	None	No
Tile.__init__	board.py	Tile	Yes
TurnDetails.__init__	game.py	TurnDetails	Yes
TurnDetails.clear_turn_details	game.py	TurnDetails	No
TurnDetails.validate_list_of_TwP	game.py	TurnDetails	No

Methods/Functions per Class:

Function/ Method	Module	Class	Used
Board.__init__	board.py	Board	Yes
Board.add_ports	board.py	Board	Yes
Board.add_intersections	board.py	Board	Yes
Board.add_roads	board.py	Board	Yes
Board.add_tiles	board.py	Board	Yes
Board.get_board	board.py	Board	Yes
Board.initialize_board	board.py	Board	Yes
Board.can_build_road_color	board.py	Board	No
Board.can_build_intersection	board.py	Board	No
Board.load_game	board.py	Board	Yes
Board.occupy_intersection	board.py	Board	No
Board.occupy_road	board.py	Board	No
Board.save_game	board.py	Board	Yes
Board.write_debug_info	board.py	Board	No
Button.__init__	gui.py	Button	Yes
EventHandler.handle_click()	event_handler.py	EventHandler	Yes
Font.initialize_fonts	gui_constants.py	Font	Yes
Game.advance_turn	game.py	Game	No
Game.distribute_rcards	game.py	Game	No
Game.handle_okn_click	game.py	Game	No
Game.handle_oky_click	game.py	Game	No
Game.roll_dice	game.py	Game	No
Game.sync_round_turn	game.py	Game	No
Game.write_debug_info	game.py	Game	No
Game.__init__	game.py	Game	Yes
Game.initialize_players	game.py	Game	Yes
GUI.__init__	gui.py	GUI	Yes
GUI.animate_element	gui.py	GUI	No
GUI.block_adjacent_in_gui()	gui.py	GUI	Yes
GUI.draw_hexagon_lines	gui.py	GUI	Yes
GUI.draw_intersections	gui.py	GUI	Yes
GUI.draw_ports	gui.py	GUI	Yes
GUI.draw_tile_values	gui.py	GUI	Yes
GUI.draw_tiles	gui.py	GUI	Yes
GUI.animate_continuous()	gui.py	GUI	Yes
GUI.occupy_city_in_gui()	gui.py	GUI	Yes
GUI.occupy_settlement_in_gui()	gui.py	GUI	Yes
GUI.occupy_road_in_gui()	gui.py	GUI	Yes
GUI.render_scoreboard_row()	gui.py	GUI	No
GUI.check_button	gui.py	GUI	Yes
GUI.check_mode	gui.py	GUI	Yes
GUI.display_fresh_board	gui.py	GUI	Yes
GUI.display_scoreboard	gui.py	GUI	Yes
GUI.draw_board_base()	gui.py	GUI	Yes
GUI.draw_all_permanent_buildings()	gui.py	GUI	Yes
GUI.draw_guidance()	gui.py	GUI	Yes
GUI.draw_guidance_text()	gui.py	GUI	Yes
GUI.print_queues()	gui.py	GUI	Yes
GUI.queue_latest_placement()	gui.py	GUI	No
GUI.handle_confirmation_click()	gui.py	GUI	Yes
GUI.set_button	gui.py	GUI	Yes
GUI.set_mode_duo	gui.py	GUI	No
GUI.update_board	gui.py	GUI	Yes
GUI.update_scoreboard()	gui.py	GUI	Yes

Function/ Method	Module	Class	Used
GUIHumanGuidance.start_settlement_phase()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.start_road_phase()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._highlight_available()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.clear()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.draw()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._compute_available_settlements()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._compute_available_road()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._show_selected()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._find_clicked_inter()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._find_clicked_road()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._place_settlement()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._place_road()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.finish_human_turn()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.on_board_click()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.on_confirmation()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.is_placing()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanPlayer.__init__	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_analysis	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_city	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_dcard	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_road	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_settlement	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_cancel	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_end_turn	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_new_game	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_quit	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_roll_dices	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_tw_b	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_tw_p	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.show_buttons_HP	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.text_buy	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.text_trade	gui_human_player.py	GUIHumanPlayer	Yes
Image.initialize_images	gui_constants.py	Image	Yes
Intersection.__init__	board.py	Intersection	Yes
cleanup_pygame	test_catan.py	None	No
convert_tile	gui.py	None	Yes
get_rcards	constants.py	None	Yes
GUIHumanPlayer.check_modes1	gui_human_player.py	None	Yes
GUIHumanPlayer.check_modes2	gui_human_player.py	None	Yes
GUIHumanPlayer.get_hp_sequence_hp	gui_human_player.py	None	Yes
GUIHumanPlayer.log_spec2	gui_human_player.py	None	Yes
intersection_probability	constants.py	None	No
main	main.py	None	Yes
setup_pygame	test_catan.py	None	No
test_board_rendering	test_catan.py	None	No
test_button_display	test_catan.py	None	No
test_game_initialization	test_catan.py	None	No
test_gui_rendering	test_catan.py	None	No
test_player_initialization	test_catan.py	None	No
Player.build_structure	player.py	Player	No
Player.can_afford	player.py	Player	No
Player.get_resource_production_probability	player.py	Player	No
Player.rcards_in_hand	player.py	Player	Yes
Player.update_trade_rates	player.py	Player	No
Player.write_debug_info	player.py	Player	No
Player.__init__	player.py	Player	Yes
Player.add_rcard	player.py	Player	No
PlayerColor.__init__	gui_constants.py	PlayerColor	Yes
ResourceCardDashboard.__init__	game.py	ResourceCardDashboard	Yes
Road.__init__	board.py	Road	Yes
Settings.__init__	game.py	Settings	Yes
Sound.initialize_sounds	gui_constants.py	Sound	Yes
StrategyDashboard.__init__	game.py	StrategyDashboard	Yes
Tile.__init__	board.py	Tile	Yes
TurnDetails.__init__	game.py	TurnDetails	Yes
TurnDetails.clear_turn_details	game.py	TurnDetails	No
TurnDetails.validate_list_of_TwP	game.py	TurnDetails	No

Methods/Functions per Module:

Function/ Method	Module	Class	Used
Board.__init__	board.py	Board	Yes
Board.add_empty_edges_and_corners	board.py	Board	Yes
Board.add_intersections	board.py	Board	Yes
Board.add_ports	board.py	Board	Yes
Board.add_roads	board.py	Board	Yes
Board.add_three_intersection_ids	board.py	Board	Yes
Board.add_three_tile_values	board.py	Board	Yes
Board.add_tiles	board.py	Board	Yes
Board.add_tiles	board.py	Board	Yes
Board.add_two_tile_attributes	board.py	Board	Yes
Board.complete_edges	board.py	Board	Yes
Board.create_list_of_roads_connected_to_intersection	board.py	Board	Yes
Board.get_board	board.py	Board	Yes
Board.initialize_board	board.py	Board	Yes
Board.is_valid_tile_value_placement	board.py	Board	No
Board.can_build_intersection_tf	board.py	Board	No
Board.can_build_road_color_tf	board.py	Board	No
Board.load_board	board.py	Board	Yes
Board.occupy_intersection	board.py	Board	No
Board.occupy_road	board.py	Board	No
Board.save_board	board.py	Board	Yes
Board.write_debug_info	board.py	Board	No
Corner.__init__	board.py	Corner	Yes
Edge.__init__	board.py	Edge	Yes
Intersection.__init__	board.py	Intersection	Yes
Road.__init__	board.py	Road	Yes
Tile.__init__	board.py	Tile	Yes
get_rcards	constants.py	None	Yes
intersection_probability	constants.py	None	No
EventHandler.handle_click()	event_handler.py	EventHandler	Yes
Game.__init__	game.py	Game	Yes
Game.initialize_players	game.py	Game	Yes
Game.advance_turn	game.py	Game	No
Game.distribute_resources	game.py	Game	No
Game.handle_okn_click	game.py	Game	No
Game.handle_oky_click	game.py	Game	No
Game.roll_dice	game.py	Game	No
Game.sync_round_turn	game.py	Game	No
Game.write_debug_info	game.py	Game	No
ResourceCardDashboard.__init__	game.py	ResourceCardDashboard	Yes
Settings.__init__	game.py	Settings	Yes
StrategyDashboard.__init__	game.py	StrategyDashboard	Yes
TurnDetails.__init__	game.py	TurnDetails	Yes
TurnDetails.clear_turn_details	game.py	TurnDetails	No
TurnDetails.validate_list_of_TwP	game.py	TurnDetails	No
Player.__init__	player.py	Player	Yes
Player.add_resource	player.py	Player	No
Player.build_structure	player.py	Player	No
Player.can_afford	player.py	Player	No
Player.get_resource_production_probability	player.py	Player	No
Player.rcards_in_hand	player.py	Player	Yes
Player.update_trade_rates	player.py	Player	No
Player.write_debug_info	player.py	Player	No
Button.__init__	gui.py	Button	Yes
GUI.__init__	gui.py	GUI	Yes
GUI.animate_element	gui.py	GUI	No
GUI.block_adjacent_in_gui()	gui.py	GUI	Yes
GUI.draw_hexagon_lines	gui.py	GUI	Yes
GUI.draw_intersections	gui.py	GUI	Yes
GUI.draw_ports	gui.py	GUI	Yes
GUI.draw_tile_values	gui.py	GUI	Yes
GUI.draw_tiles	gui.py	GUI	Yes

Function/ Method	Module	Class	Used
GUI.animate_continuous()	gui.py	GUI	Yes
GUI.occupy_city_in_gui()	gui.py	GUI	Yes
GUI.occupy_settlement_in_gui()	gui.py	GUI	Yes
GUI.occupy_road_in_gui()	gui.py	GUI	Yes
GUI.render_scoreboard_row()	gui.py	GUI	No
GUI.check_button	gui.py	GUI	Yes
GUI.check_mode	gui.py	GUI	Yes
GUI.display_fresh_board	gui.py	GUI	Yes
GUI.display_scoreboard	gui.py	GUI	Yes
GUI.draw_board_base()	gui.py	GUI	Yes
GUI.draw_all_permanent_buildings()	gui.py	GUI	Yes
GUI.draw_guidance()	gui.py	GUI	Yes
GUI.draw_guidance_text()	gui.py	GUI	Yes
GUI.print_queues()	gui.py	GUI	Yes
GUI.queue_latest_placement()	gui.py	GUI	No
GUI.handle_confirmation_click()	gui.py	GUI	Yes
GUI.set_button	gui.py	GUI	Yes
GUI.set_mode_duo	gui.py	GUI	No
GUI.update_board	gui.py	GUI	Yes
GUI.update_scoreboard()	gui.py	GUI	Yes
convert_tile	gui.py	None	Yes
Font.initialize_fonts	gui_constants.py	Font	Yes
Image.initialize_images	gui_constants.py	Image	Yes
PlayerColor.__init__	gui_constants.py	PlayerColor	Yes
Sound.initialize_sounds	gui_constants.py	Sound	Yes
GUIHumanGuidance.start_settlement_phase()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.start_road_phase()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._highlight_available()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.clear()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.draw()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._compute_available_settlements()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._compute_available_roads()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._show_selected()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._find_clicked_inter()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._find_clicked_road()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._place_settlement()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._place_road()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance._finish_human_turn()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.on_board_click()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.on_confirmation()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanGuidance.is_placing()	gui_guidance.py	GUIHumanGuidance	Yes
GUIHumanPlayer.__init__	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_analysis	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_city	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_dcard	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_road	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_buy_settlement	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_cancel	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_end_turn	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_new_game	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_quit	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_roll_dice	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_tw	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.button_twp	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.show_buttons_HP	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.text_buy	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.text_trade	gui_human_player.py	GUIHumanPlayer	Yes
GUIHumanPlayer.check_modes1	gui_human_player.py	None	Yes
GUIHumanPlayer.check_modes2	gui_human_player.py	None	Yes
GUIHumanPlayer.get_hp_sequence_hp	gui_human_player.py	None	Yes
GUIHumanPlayer.log_spec2	gui_human_player.py	None	Yes
main	main.py	None	Yes
cleanup_pygame	test_catan.py	None	Yes*
setup_pygame	test_catan.py	None	Yes*
test_board_rendering	test_catan.py	None	Yes*
test_button_display	test_catan.py	None	Yes*
test_game_initialization	test_catan.py	None	Yes*
test_gui_rendering	test_catan.py	None	Yes*
test_player_initialization	test_catan.py	None	Yes*
test_save_load_board	test_catan.py	None	Yes*
Yes* = When testing			

Methods/Functions per Module including description:

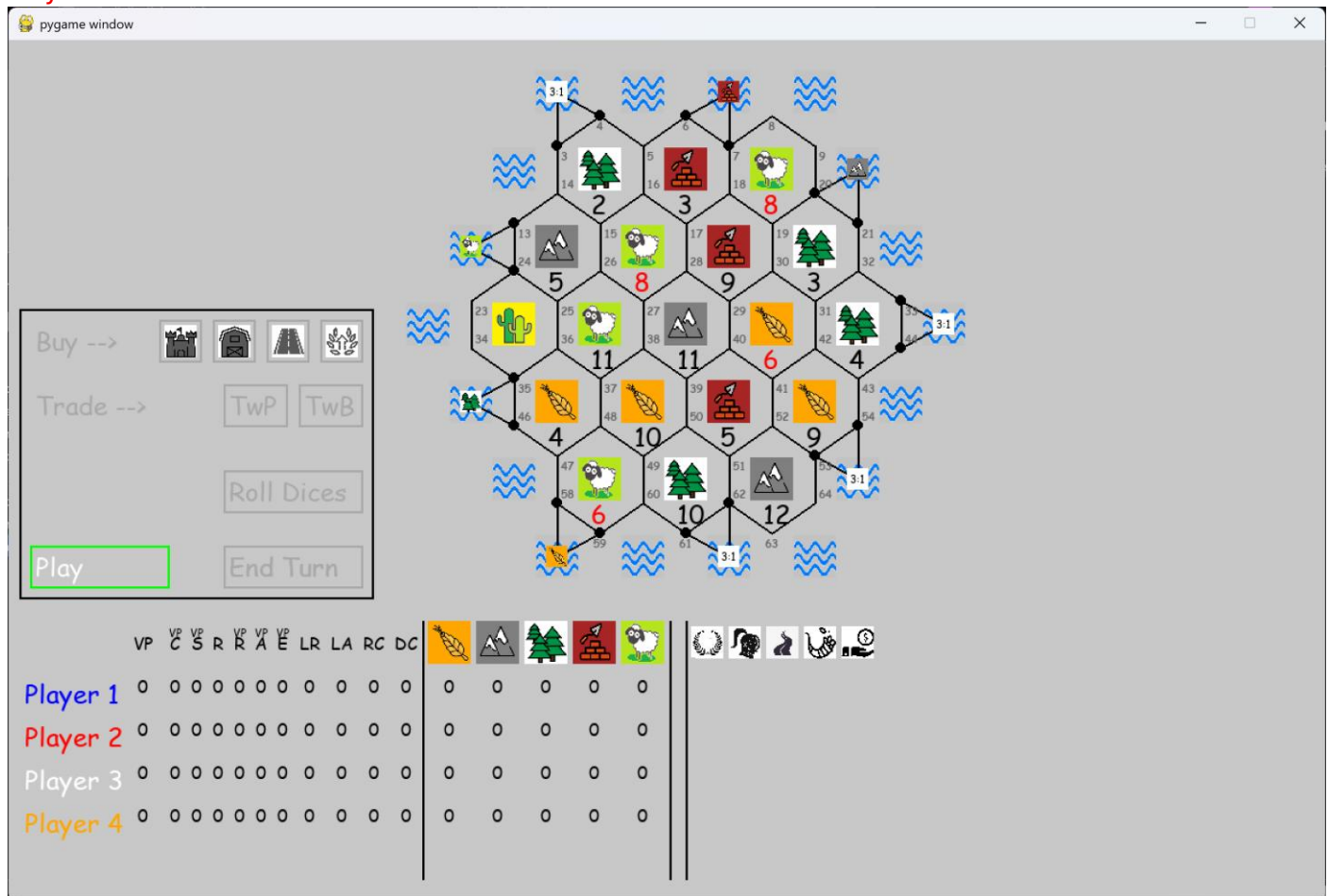
Function/ Method	Module	Class	Description	Used
Module board.py				
__init__	board.py	Board	Initializes the Board instance with a configuration (e.g. "Base_Random").	Yes
add_empty_edges_and_corners		Board	Initializes empty edges and corners for each tile.	Yes
add_intersections	board.py	Board	Adds intersections with tile associations.	
add_ports	board.py	Board	Adds ports to intersections.	
add_roads	board.py	Board	Adds roads to the board.	
add_three_intersection_ids		Board	Adds neighboring intersection IDs for each intersection based on connected roads.	
add_three_tile_values		Board	Adds tile values for each intersection's neighboring tiles.	
add_tiles	board.py	Board	Adds tiles to the board based on BOARD_LAYOUT.	
add_two_tile_attributes		Board	Adds tile attributes to roads based on adjacent tiles	
complete_edges		Board	Assigns road IDs to tile edges based on corner intersections.	No
create_list_of_roads_connected_to_intersection		Board	Creates a list of roads connected to each intersection.	No
get_board	board.py	Board	Generates a random board.	
initialize_board	board.py	Board	Initializes the board's tiles, intersections and ports based on the configuration.	No
is_valid_tile_value_placement		Board	Checks if tiles with values 6 or 8 are not adjacent.	No
can_build_intersection_tf	board.py	Board	Checks if a settlement or city can be built at an intersection.	
can_build_road_color_tf	board.py	Board	Checks if a road can be built by a player.	No
load_board		Board	Load a playboard, having specific tiles and harbors.	
occupy_intersection	board.py	Board	Occupies an intersection with a settlement or city.	
occupy_road	board.py	Board	Occupies a road.	Yes
save_board		Board	Save a playboard.	Yes
write_debug_info	board.py	Board	Writes board attributes to FILENAME_MG for debugging.	Yes
__init__		Corner	Initializes a Corner with location, kind, color, port_type, and intersection.	Yes
__init__		Edge	Initializes an Edge with location, kind, color, and road.	Yes
__init__	board.py	Intersection	Initializes an Intersection.	Yes
__init__	board.py	Road	Initializes a Road.	Yes
__init__	board.py	Tile	Initializes a Tile.	Yes
Module player.py				
__init__	player.py	Player	Initializes a player with an ID.	No
add_rcard		Player	Adds resource cards to the player's hand.	
build_structure	player.py	Player	Builds a structure on the board.	No
can_afford	player.py	Player	Checks if the player can afford to build a structure.	No
get_resource_production_probability	player.py	Player	Calculates the total resource production probability for each resource.	No
rcards_in_hand		Player	Retrieves resource cards in player's hand.	No
update_trade_rates	player.py	Player	Updates trade rates based on port access.	
write_debug_info	player.py	Player	Writes player attributes to FILENAME_MG for debugging.	Yes
Module game.py				
__init__	game.py	Game	Initializes the Game instance with: sequence number, ID, phase, state and board	No
initialize_players	game.py	Game	Initializes players for the game.	No
advance_turn	game.py	Game	Advances to the next player's turn.	No
distribute_resources	game.py	Game	Distributes resources based on dice roll.	No
handle_okn_click	game.py	Game	Handles OKN button click for human player/operator.	No
handle_oky_click	game.py	Game	Handles OKY button click for human player/operator.	No
roll_dice	game.py	Game	Simulates rolling two dice.	No
sync_round_turn	game.py	Game	Syncs round and turn with myturn.	Yes
write_debug_info	game.py	Game	Writes game attributes to FILENAME_MG for debugging.	Yes
__init__	game.py	ResourceCardDashboard	Initializes a ResourceCardDashboard.	Yes
__init__	game.py	Settings	Initializes Settings.	No
__init__	game.py	StrategyDashboard	Initializes a StrategyDashboard.	No
__init__	game.py	TurnDetails	Initializes TurnDetails.	Yes
clear_turn_details	game.py	TurnDetails	Clears all turn details to initial values.	Yes
validate_list_of_TwP	game.py	TurnDetails	Validates the list of Trade with Players (TwP).	Yes
Module constants.py				
get_rcards	constants.py	None	Generates a dictionary of resource card costs for building actions.	Yes
intersection_probability	constants.py	None	Calculates the probability of a dice roll for an intersection.	No
Module event_handler.py				
handle_click()	constants.py	EventHandler	Main entry point for all mouse clicks in the game: 1) OKY/OKN, 2) "Play" button,3) delegates to on_board_click() within gui_guidance.	Yes

Fu+B60:E112nction/ Method	Module	Class	Description	Used
Module gui.py				
__init__	gui.py	Button	Initializes a button with name and display state.	Yes
__init__	gui.py	GUI	Initializes GUI with round and turn.	Yes
_animate_element	gui.py	GUI	Animates a board element (stub).	No
_block_adjacent_in_gui()	gui.py	GUI	Renders black dots on adjacent intersections permanently blocked	Yes
_draw_hexagon_lines	gui.py	GUI	Draws hexagon lines connecting intersections.	Yes
_draw_intersections	gui.py	GUI	Draws intersections with bold IDs.	Yes
_draw_ports	gui.py	GUI	Draws port icons, circles and lines.	Yes
_draw_tile_values	gui.py	GUI	Draws number chits on tiles.	Yes
_draw_tiles	gui.py	GUI	Draws hexagonal tiles.	Yes
_occupy_city_in_gui()	gui.py	GUI	Blits the CITY_{color} image (30×30) at the intersection position.	Yes
_occupy_settlement_in_gui()	gui.py	GUI	Blits the SETTLEMENT_{color} image (30×30) at the intersection position.	Yes
_occupy_road_in_gui()	gui.py	GUI	Draws a thick coloured line between the two intersection positions.	Yes
_render_scoreboard_row()	gui.py	GUI	Renders one complete player row in the scoreboard.	No
animate_continuous()	gui.py	GUI	Runs the continuous pulsing animation on the current animation queue.	Yes
check_button	gui.py	GUI	Checks if a button is set to display.	Yes
check_mode	gui.py	GUI	Checks if a mode is active (stub).	Yes
display_fresh_board	gui.py	GUI	Renders the initial board and optional scoreboard.	Yes
display_scoreboard	gui.py	GUI	Placeholder for scoreboard rendering.	Yes
draw_board_base()	gui.py	GUI	Draws the static empty board (tiles, ports, ...) without any buildings.	Yes
draw_all_permanent_buildings()	gui.py	GUI	Draws every currently placed road, settlement and city.	Yes
draw_guidance()	gui.py	GUI	Shows pulsing highlights, selected items and OKY/OKN icons.	Yes
draw_guidance_text()	gui.py	GUI	Provide instructional text (supports single string or list).	Yes
print_queues()	gui.py	GUI	Logs the contents of the animation queue for debugging.	Yes
queue_latest_placement()	gui.py	GUI	Populates the animation queue with the most recent settlement/road.	No
handle_confirmation_click()	gui.py	GUI	Checks whether the click landed on the dynamic OKY or OKN icon.	Yes
set_button	gui.py	GUI	Sets a button's display state.	Yes
set_mode_duo	gui.py	GUI	Sets two modes (stub).	No
update_board	gui.py	GUI	Updates the board display based on an action.	Yes
update_scoreboard()	gui.py	GUI	Renders the complete scoreboard.	Yes
convert_tile	gui.py	None	Converts a tile ID to GUI midpoint coordinates.	Yes
Module gui_game.py				
__init__	gui_game.py	GUIGame	Initializes the GUIGame.	Yes
_render_scoreboard_row	gui_game.py	GUIGame	Renders a single row of the scoreboard.	Yes
update_scoreboard	gui_game.py	GUIGame	Renders the scoreboard with game statistics for players.	Yes
Module gui_guidance.py				
GUIHumanGuidance.start_settlement_phase()	gui_guidance.py	GUIHumanGuidance	Computes legal intersections, enables pulsing highlights, shows guidance text, and disables the Play button.	Yes
GUIHumanGuidance.start_road_phase()	gui_guidance.py	GUIHumanGuidance	Clears queue, computes legal roads from the selected settlement, starts pulsing, and updates guidance text.	Yes
GUIHumanGuidance._highlight_available()	gui_guidance.py	GUIHumanGuidance	Adds pulsing coloured circles to the central animation queue for visual feedback during selection phases.	Yes
GUIHumanGuidance.clear()	gui_guidance.py	GUIHumanGuidance	Resets HumanGuidance to IDLE state, clears selection data and confirmation icons, and re-enables the Play button.	Yes
GUIHumanGuidance.draw()	gui_guidance.py	GUIHumanGuidance	Renders pulsing highlights (while choosing), selected item, and OKY/OKN confirmation icons; called every frame from the main loop.	Yes
GUIHumanGuidance._compute_available_settlements()	gui_guidance.py	GUIHumanGuidance	Builds list of all valid settlement intersections using game.can_build_intersection_tf() (respects distance rule and road connection).	Yes
GUIHumanGuidance._compute_available_roads()	gui_guidance.py	GUIHumanGuidance	Builds list of valid roads connected to the currently selected settlement using can_build_road_color_tf().	Yes
GUIHumanGuidance._show_selected()	gui_guidance.py	GUIHumanGuidance	Temporarily draws only the currently selected settlement or road (used after click but before OKY confirmation).	Yes
GUIHumanGuidance._find_clicked_inter()	gui_guidance.py	GUIHumanGuidance	Returns the intersection ID if the click is inside its hit area (distance < 15 pixels), otherwise None.	Yes
GUIHumanGuidance._find_clicked_road()	gui_guidance.py	GUIHumanGuidance	Returns the road tuple if the click is near its midpoint (distance < 15 pixels) and the road is available.	Yes
GUIHumanGuidance._place_settlement()	gui_guidance.py	GUIHumanGuidance	Permanently places the settlement on the board, distributes initial resources (if round == -1), and updates dashboards.	Yes
GUIHumanGuidance._place_road()	gui_guidance.py	GUIHumanGuidance	Permanently places the selected road on the board.	Yes
GUIHumanGuidance._finish_human_turn()	gui_guidance.py	GUIHumanGuidance	Finalises the human turn: redraws permanent buildings, enables the Play button, clears guidance state, and advances the turn when Play is clicked.	Yes
GUIHumanGuidance.on_board_click()	gui_guidance.py	GUIHumanGuidance	Handles clicks during placement: detects valid settlement or road click, changes state and shows confirmation icons.	Yes
GUIHumanGuidance.on_confirmation()	gui_guidance.py	GUIHumanGuidance	Processes OKY/OKN clicks: on OKY performs placement and state transition; on OKN returns to previous selection phase.	Yes
GUIHumanGuidance.is_placing()	gui_guidance.py	GUIHumanGuidance	Returns True while any placement phase is active (used by GUIHumanPlayer to disable/enable buttons).	Yes

Function/ Method	Module	Class	Description	Used
Module gui_human_player.py				
__init__	gui_human_player.py	GUIHumanPlayer	Initializes the GUIHumanPlayer.	Yes
button_analysis	gui_human_player.py	GUIHumanPlayer	Renders "Analysis" button.	Yes
button_buy_city	gui_human_player.py	GUIHumanPlayer	Renders "Buy City" button with 30x30 image at (145, 265).	Yes
button_buy_dcard	gui_human_player.py	GUIHumanPlayer	Renders "Buy DCard" button with 30x30 image at (295, 265).	Yes
button_buy_road	gui_human_player.py	GUIHumanPlayer	Renders "Buy Road" button with 30x30 image at (245, 265).	Yes
button_buy_settlement	gui_human_player.py	GUIHumanPlayer	Renders "Buy Settlement" button with 30x30 image at (195, 265).	Yes
button_cancel	gui_human_player.py	GUIHumanPlayer	Renders "Cancel" button.	Yes
button_end_turn	gui_human_player.py	GUIHumanPlayer	Renders "End Turn" button.	Yes
button_new_game	gui_human_player.py	GUIHumanPlayer	Renders "New Game" button.	Yes
button_quit	gui_human_player.py	GUIHumanPlayer	Renders "Quit" button.	Yes
button_roll_dices	gui_human_player.py	GUIHumanPlayer	Renders "Roll Dices" button.	Yes
button_twob	gui_human_player.py	GUIHumanPlayer	Renders "Trade with Bank" button.	Yes
button_two	gui_human_player.py	GUIHumanPlayer	Renders "Trade with Player" button.	Yes
show_buttons_HP	gui_human_player.py	GUIHumanPlayer	Renders all buttons for human player actions.	Yes
text_buy	gui_human_player.py	GUIHumanPlayer	Renders "Buy -->" text.	Yes
text_trade	gui_human_player.py	GUIHumanPlayer	Renders "Trade -->" text.	Yes
check_modes1	gui_human_player.py	None	Checks if any button is displayed.	Yes
check_modes2	gui_human_player.py	None	Checks if specific modes are active.	Yes
get_hp_sequence_hp	gui_human_player.py	None	Returns turn sequence for the human player.	Yes
log_spec2	gui_human_player.py	None	Logs button modes to FILENAME_SPEC2.	Yes
Module gui_constants.py				
Font.initialize_fonts	gui_constants.py	Font	Initializes fonts for rendering.	Yes
Image.initialize_images	gui_constants.py	Image	Loads and scales images into the IMAGES dictionary.	Yes
PlayerColor.__init__	gui_constants.py	PlayerColor	Initializes a PlayerColor (auto-generated by Enum).	Yes
Sound.initialize_sounds	gui_constants.py	Sound	Loads sounds into the SOUNDS dictionary.	Yes
Module test_catan.py				
cleanup_pygame	test_catan.py	None	Quits Pygame and cleans up.	No
setup_pygame	test_catan.py	None	Sets up the Pygame display.	No
test_board_rendering	test_catan.py	None	Tests board rendering functionality.	No
test_button_display	test_catan.py	None	Tests button display functionality.	No
test_game_initialization	test_catan.py	None	Tests the initialization of the Game class.	No
test_gui_rendering	test_catan.py	None	Tests the GUI rendering components.	
test_player_initialization	test_catan.py	None	Tests the initialization of Player instances.	No
test_save_load_board		None	Tests saving and loading a random board.	No

Annex 5: Example

Playboard:



Data (captured in ..._MG.txt):

Re-try due to incorrect placement of tiles with values 6 or 8:

```
board.py | _is_valid_tile_value_placement | Invalid: Intersection 16 has 2 tiles with values 6 or 8: [6, 0, 8]
board.py | _get_board | Attempt 1 failed: Retrying tile value placement
board.py | _is_valid_tile_value_placement | Invalid: Intersection 18 has 2 tiles with values 6 or 8: [3, 8, 6]
board.py | _get_board | Attempt 2 failed: Retrying tile value placement
board.py | _is_valid_tile_value_placement | Invalid: Intersection 62 has 2 tiles with values 6 or 8: [8, 0, 8]
board.py | _get_board | Attempt 3 failed: Retrying tile value placement
board.py | _is_valid_tile_value_placement | Invalid: Intersection 38 has 2 tiles with values 6 or 8: [3, 6, 8]
board.py | _get_board | Attempt 4 failed: Retrying tile value placement
board.py | _is_valid_tile_value_placement | Invalid: Intersection 14 has 2 tiles with values 6 or 8: [0, 6, 8]
board.py | _get_board | Attempt 5 failed: Retrying tile value placement
board.py | _is_valid_tile_value_placement | Invalid: Intersection 18 has 2 tiles with values 6 or 8: [6, 8, 9]
board.py | _get_board | Attempt 6 failed: Retrying tile value placement
board.py | _is_valid_tile_value_placement | Invalid: Intersection 40 has 2 tiles with values 6 or 8: [8, 8, 9]
board.py | _get_board | Attempt 7 failed: Retrying tile value placement
board.py | _is_valid_tile_value_placement | Invalid: Intersection 38 has 2 tiles with values 6 or 8: [0, 6, 8]
board.py | _get_board | Attempt 8 failed: Retrying tile value placement
```

Playboard data:

```
board.py | add_roads | Added 72 roads
```

```
board.py | write_debug_info | Board
```

```
Board Attributes: {'board_name': 'Base_Random', 'round': -2, 'turn': 1}
```

```
Intersection IDs: [3, 4, 5, 6, 7, 8, 9, 13, 14, 15, 16, 17, 18, 19, 20, 21, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 46, 47, 48, 49, 50, 51, 52, 53, 54, 58, 59, 60, 61, 62, 63, 64]
```

```
Road IDs: [(3, 4), (5, 6), (4, 5), (7, 8), (6, 7), (8, 9), (3, 14), (13, 14), (5, 16), (15, 16), (14, 15), (7, 18), (17, 18), (16, 17), (9, 20), (19, 20), (18, 19), (20, 21), (13, 24), (23, 24), (15, 26), (25, 26), (24, 25), (17, 28), (27, 28), (26, 27), (19, 30), (29, 30), (28, 29), (21, 32), (31, 32), (30, 31), (32, 33), (23, 34), (25, 36), (35, 36), (34, 35), (27, 38), (37, 38), (36, 37), (29, 40), (39, 40), (38, 39), (31, 42), (41, 42), (40, 41), (33, 44), (43, 44), (42, 43), (35, 46), (37, 48), (47, 48), (46, 47), (39, 50), (49, 50), (48, 49), (41, 52), (51, 52), (50, 51), (43, 54), (53, 54), (52, 53), (47, 58), (49, 60), (59, 60), (58, 59), (51, 62), (61, 62), (60, 61), (53, 64), (63, 64), (62, 63)]
```

```
Tile IDs: [2, 3, 4, 5, 8, 9, 10, 11, 12, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 34, 35, 36, 37, 38, 41, 42, 43, 44]
```

Tile data including Edges and Corners:

board.py | write_debug_info | Tiles

- [illegible]

- Edge (road=(0, 0)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
- Edge (road=(0, 0)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
- Corners:
 - Corner (intersection=0): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=3): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 3}
 - Corner (intersection=14): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 14}
 - Corner (intersection=13): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Wool', 'intersection': 13}
 - Corner (intersection=0): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
- Tile Attributes: {'id': 9, 'type': 'Forest', 'value': 2, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(4, 5)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (4, 5)}
 - Edge (road=(5, 16)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (5, 16)}
 - Edge (road=(15, 16)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (15, 16)}
 - Edge (road=(14, 15)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (14, 15)}
 - Edge (road=(3, 14)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (3, 14)}
 - Edge (road=(3, 4)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (3, 4)}
 - Corners:
 - Corner (intersection=4): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 4}
 - Corner (intersection=5): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 5}
 - Corner (intersection=16): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 16}
 - Corner (intersection=15): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 15}
 - Corner (intersection=14): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 14}
 - Corner (intersection=3): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 3}
- Tile Attributes: {'id': 10, 'type': 'Hill', 'value': 3, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(6, 7)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (6, 7)}
 - Edge (road=(7, 18)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (7, 18)}
 - Edge (road=(17, 18)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (17, 18)}
 - Edge (road=(16, 17)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (16, 17)}
 - Edge (road=(5, 16)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (5, 16)}
 - Edge (road=(5, 6)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (5, 6)}
 - Corners:
 - Corner (intersection=6): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Brick', 'intersection': 6}
 - Corner (intersection=7): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Brick', 'intersection': 7}
 - Corner (intersection=18): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 18}
 - Corner (intersection=17): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 17}
 - Corner (intersection=16): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 16}
 - Corner (intersection=5): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 5}
- Tile Attributes: {'id': 11, 'type': 'Pasture', 'value': 8, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(8, 9)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (8, 9)}
 - Edge (road=(9, 20)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (9, 20)}
 - Edge (road=(19, 20)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (19, 20)}
 - Edge (road=(18, 19)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (18, 19)}
 - Edge (road=(7, 18)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (7, 18)}
 - Edge (road=(7, 8)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (7, 8)}
 - Corners:
 - Corner (intersection=8): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 8}
 - Corner (intersection=9): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 9}
 - Corner (intersection=20): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Ore', 'intersection': 20}
 - Corner (intersection=19): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 19}
 - Corner (intersection=18): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 18}
 - Corner (intersection=7): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Brick', 'intersection': 7}
- Tile Attributes: {'id': 12, 'type': 'Sea', 'value': 0, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(0, 0)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(20, 21)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (20, 21)}
 - Edge (road=(9, 20)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (9, 20)}
 - Edge (road=(0, 0)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Corners:
 - Corner (intersection=0): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=21): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Ore', 'intersection': 21}
 - Corner (intersection=20): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Ore', 'intersection': 20}
 - Corner (intersection=9): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 9}
- Tile: None - [Note: this is Tile\[13\]](#)
- Tile Attributes: {'id': 14, 'type': 'Sea', 'value': 0, 'color': 'Blank', 'occupied_tf': False}
 - Edges:

- [illegible]

- Corner (intersection=19): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 19}
- Tile Attributes: {'id': 19, 'type': 'Sea', 'value': 0, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(0, 0)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(32, 33)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (32, 33)}
 - Edge (road=(21, 32)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (21, 32)}
 - Edge (road=(0, 0)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Corners:
 - Corner (intersection=0): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=33): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 33}
 - Corner (intersection=32): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 32}
 - Corner (intersection=21): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Ore', 'intersection': 21}
- Tile Attributes: {'id': 20, 'type': 'Sea', 'value': 0, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(0, 0)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(23, 34)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (23, 34)}
 - Edge (road=(0, 0)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Corners:
 - Corner (intersection=0): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=23): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 23}
 - Corner (intersection=34): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 34}
 - Corner (intersection=0): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
- Tile Attributes: {'id': 21, 'type': 'Desert', 'value': 0, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(24, 25)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (24, 25)}
 - Edge (road=(25, 36)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (25, 36)}
 - Edge (road=(35, 36)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (35, 36)}
 - Edge (road=(34, 35)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (34, 35)}
 - Edge (road=(23, 34)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (23, 34)}
 - Edge (road=(23, 24)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (23, 24)}
 - Corners:
 - Corner (intersection=24): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Wool', 'intersection': 24}
 - Corner (intersection=25): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 25}
 - Corner (intersection=36): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 36}
 - Corner (intersection=35): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Wood', 'intersection': 35}
 - Corner (intersection=34): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 34}
 - Corner (intersection=23): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 23}
- Tile Attributes: {'id': 22, 'type': 'Pasture', 'value': 11, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(26, 27)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (26, 27)}
 - Edge (road=(27, 38)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (27, 38)}
 - Edge (road=(37, 38)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (37, 38)}
 - Edge (road=(36, 37)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (36, 37)}
 - Edge (road=(25, 36)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (25, 36)}
 - Edge (road=(25, 26)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (25, 26)}
 - Corners:
 - Corner (intersection=26): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 26}
 - Corner (intersection=27): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 27}
 - Corner (intersection=38): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 38}
 - Corner (intersection=37): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 37}
 - Corner (intersection=36): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 36}
 - Corner (intersection=25): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 25}
- Tile Attributes: {'id': 23, 'type': 'Mountain', 'value': 11, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(28, 29)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (28, 29)}
 - Edge (road=(29, 40)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (29, 40)}
 - Edge (road=(39, 40)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (39, 40)}
 - Edge (road=(38, 39)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (38, 39)}
 - Edge (road=(27, 38)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (27, 38)}
 - Edge (road=(27, 28)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (27, 28)}
 - Corners:
 - Corner (intersection=28): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 28}
 - Corner (intersection=29): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 29}

- [illegible]

- [illegible]

- Edge (road=(0, 0)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
- Edge (road=(0, 0)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
- Edge (road=(0, 0)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
- Edge (road=(0, 0)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
- Corners:
 - Corner (intersection=46): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Wood', 'intersection': 46}
 - Corner (intersection=47): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 47}
 - Corner (intersection=58): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Wheat', 'intersection': 58}
 - Corner (intersection=0): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
- Tile Attributes: {'id': 35, 'type': 'Pasture', 'value': 6, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(48, 49)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (48, 49)}
 - Edge (road=(49, 60)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (49, 60)}
 - Edge (road=(59, 60)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (59, 60)}
 - Edge (road=(58, 59)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (58, 59)}
 - Edge (road=(47, 58)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (47, 58)}
 - Edge (road=(47, 48)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (47, 48)}
 - Corners:
 - Corner (intersection=48): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 48}
 - Corner (intersection=49): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 49}
 - Corner (intersection=60): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 60}
 - Corner (intersection=59): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Wheat', 'intersection': 59}
 - Corner (intersection=58): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': '2:1 Wheat', 'intersection': 47}
 - Corner (intersection=47): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 47}
- Tile Attributes: {'id': 36, 'type': 'Forest', 'value': 10, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(50, 51)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (50, 51)}
 - Edge (road=(51, 62)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (51, 62)}
 - Edge (road=(61, 62)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (61, 62)}
 - Edge (road=(60, 61)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (60, 61)}
 - Edge (road=(49, 60)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (49, 60)}
 - Edge (road=(49, 50)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (49, 50)}
 - Corners:
 - Corner (intersection=50): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 50}
 - Corner (intersection=51): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 51}
 - Corner (intersection=62): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 62}
 - Corner (intersection=61): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 61}
 - Corner (intersection=60): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 60}
 - Corner (intersection=49): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 49}
- Tile Attributes: {'id': 37, 'type': 'Mountain', 'value': 12, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(52, 53)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (52, 53)}
 - Edge (road=(53, 64)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (53, 64)}
 - Edge (road=(63, 64)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (63, 64)}
 - Edge (road=(62, 63)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (62, 63)}
 - Edge (road=(51, 62)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (51, 62)}
 - Edge (road=(51, 52)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (51, 52)}
 - Corners:
 - Corner (intersection=52): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 52}
 - Corner (intersection=53): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 53}
 - Corner (intersection=64): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 64}
 - Corner (intersection=63): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 63}
 - Corner (intersection=62): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 62}
 - Corner (intersection=51): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 51}
- Tile Attributes: {'id': 38, 'type': 'Sea', 'value': 0, 'color': 'Blank', 'occupied_tf': False}
 - Edges:
 - Edge (road=(0, 0)): {'location': 'NE', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'E', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'SE', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(0, 0)): {'location': 'SW', 'kind': 'Blank', 'color': 'Blank', 'road': (0, 0)}
 - Edge (road=(53, 64)): {'location': 'W', 'kind': 'Blank', 'color': 'Blank', 'road': (53, 64)}
 - Edge (road=(53, 54)): {'location': 'NW', 'kind': 'Blank', 'color': 'Blank', 'road': (53, 54)}
 - Corners:
 - Corner (intersection=54): {'location': 'N', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 54}
 - Corner (intersection=0): {'location': 'EH', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'EL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=0): {'location': 'S', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 0}
 - Corner (intersection=64): {'location': 'WL', 'kind': 'Blank', 'color': 'Blank', 'port_type': 'Blank', 'intersection': 64}
 - Corner (intersection=53): {'location': 'WH', 'kind': 'Blank', 'color': 'Blank', 'port_type': '3:1', 'intersection': 53}
- Tile: None - [Note: this is Tile\[39\]](#)

- [illegible]

Intersection data:

board.py | write_debug_info | Intersections

- Intersection: None - [Note: this is Intersection\[0\]](#), which was added to make sure Intersection[1] corresponds with Intersection.id == 1
- Intersection: None - [Note: this is Intersection\[1\]](#)
- Intersection: None - [Note: this is Intersection\[2\]](#)
- Intersection Attributes: {'id': 3, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [8, 2, 9], 'three_tile_probabilities': [0, 0, 1], 'three_tile_types': ['Sea', 'Sea', 'Forest'], 'three_tile_values': [], 'all_tile_types': [0, 0, 1, 0, 0], 'all_tile_probabilities': [0, 0, 1, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(3, 4), (3, 14)], 'three_intersection_ids': [4, 14], 'port_tf': True, 'port_type': '3:1'}
- Intersection Attributes: {'id': 4, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [2, 9, 3], 'three_tile_probabilities': [0, 1, 0], 'three_tile_types': ['Sea', 'Forest', 'Sea'], 'three_tile_values': [0, 2, 0], 'all_tile_types': [0, 0, 1, 0, 0], 'all_tile_probabilities': [0, 0, 1, 0, 0], 'all_tile_values': [1, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(3, 4), (4, 5)], 'three_intersection_ids': [3, 5], 'port_tf': True, 'port_type': '3:1'}
- Intersection Attributes: {'id': 5, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [9, 3, 10], 'three_tile_probabilities': [1, 0, 2], 'three_tile_types': ['Forest', 'Sea', 'Hill'], 'three_tile_values': [], 'all_tile_types': [0, 0, 1, 1, 0], 'all_tile_probabilities': [0, 0, 1, 2, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(5, 6), (4, 5), (5, 16)], 'three_intersection_ids': [6, 4, 16], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 6, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [3, 10, 4], 'three_tile_probabilities': [0, 2, 0], 'three_tile_types': ['Sea', 'Hill', 'Sea'], 'three_tile_values': [0, 3, 0], 'all_tile_types': [0, 0, 0, 1, 0], 'all_tile_probabilities': [0, 0, 0, 2, 0], 'all_tile_values': [0, 1, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(5, 6), (6, 7)], 'three_intersection_ids': [5, 7], 'port_tf': True, 'port_type': '2:1 Brick'}
- Intersection Attributes: {'id': 7, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [10, 4, 11], 'three_tile_probabilities': [2, 0, 5], 'three_tile_types': ['Hill', 'Sea', 'Pasture'], 'three_tile_values': [], 'all_tile_types': [0, 0, 0, 1, 1], 'all_tile_probabilities': [0, 0, 0, 2, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(7, 8), (6, 7), (7, 18)], 'three_intersection_ids': [8, 6, 18], 'port_tf': True, 'port_type': '2:1 Brick'}
- Intersection Attributes: {'id': 8, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [4, 11, 5], 'three_tile_probabilities': [0, 5, 0], 'three_tile_types': ['Sea', 'Pasture', 'Sea'], 'three_tile_values': [0, 8, 0], 'all_tile_types': [0, 0, 0, 0, 1], 'all_tile_probabilities': [0, 0, 0, 0, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 1, 0, 0, 0], 'three_roads': [(7, 8), (8, 9)], 'three_intersection_ids': [7, 9], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 9, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [11, 5, 12], 'three_tile_probabilities': [5, 0, 0], 'three_tile_types': ['Pasture', 'Sea', 'Sea'], 'three_tile_values': [], 'all_tile_types': [0, 0, 0, 0, 1], 'all_tile_probabilities': [0, 0, 0, 0, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(8, 9), (9, 20)], 'three_intersection_ids': [8, 20], 'port_tf': False, 'port_type': 'Blank'}
- Intersection: None - [Note: this is Intersection\[10\]](#)
- Intersection: None - [Note: this is Intersection\[11\]](#)
- Intersection: None - [Note: this is Intersection\[12\]](#)
- Intersection Attributes: {'id': 13, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [14, 8, 15], 'three_tile_probabilities': [0, 0, 4], 'three_tile_types': ['Sea', 'Sea', 'Mountain'], 'three_tile_values': [], 'all_tile_types': [0, 1, 0, 0, 0], 'all_tile_probabilities': [0, 4, 0, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(13, 14), (13, 24)], 'three_intersection_ids': [14, 24], 'port_tf': True, 'port_type': '2:1 Wool'}
- Intersection Attributes: {'id': 14, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [8, 15, 9], 'three_tile_probabilities': [0, 4, 1], 'three_tile_types': ['Sea', 'Mountain', 'Forest'], 'three_tile_values': [0, 5, 2], 'all_tile_types': [0, 1, 1, 0, 0], 'all_tile_probabilities': [0, 4, 1, 0, 0], 'all_tile_values': [1, 0, 0, 1, 0, 0, 0, 0, 0, 0], 'three_roads': [(3, 14), (13, 14), (14, 15)], 'three_intersection_ids': [3, 13, 15], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 15, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [15, 9, 16], 'three_tile_probabilities': [4, 1, 5], 'three_tile_types': ['Mountain', 'Forest', 'Pasture'], 'three_tile_values': [], 'all_tile_types': [0, 1, 1, 0, 1], 'all_tile_probabilities': [0, 4, 1, 0, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(15, 16), (14, 15), (15, 26)], 'three_intersection_ids': [16, 14, 26], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 16, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [9, 16, 10], 'three_tile_probabilities': [1, 5, 2], 'three_tile_types': ['Forest', 'Pasture', 'Hill'], 'three_tile_values': [2, 8, 3], 'all_tile_types': [0, 0, 1, 1, 1], 'all_tile_probabilities': [0, 0, 1, 2, 5], 'all_tile_values': [1, 1, 0, 0, 0, 0, 1, 0, 0, 0], 'three_roads': [(5, 16), (15, 16), (16, 17)], 'three_intersection_ids': [5, 15, 17], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 17, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [16, 10, 17], 'three_tile_probabilities': [5, 2, 4], 'three_tile_types': ['Pasture', 'Hill', 'Hill'], 'three_tile_values': [], 'all_tile_types': [0, 0, 0, 2, 1], 'all_tile_probabilities': [0, 0, 0, 6, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(17, 18), (16, 17), (17, 28)], 'three_intersection_ids': [18, 16, 28], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 18, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [10, 17, 11], 'three_tile_probabilities': [2, 4, 5], 'three_tile_types': ['Hill', 'Hill', 'Pasture'], 'three_tile_values': [3, 9, 8], 'all_tile_types': [0, 0, 0, 2, 1], 'all_tile_probabilities': [0, 0, 0, 6, 5], 'all_tile_values': [0, 1, 0, 0, 0, 0, 1, 1, 0, 0], 'three_roads': [(7, 18), (17, 18), (18, 19)], 'three_intersection_ids': [7, 17, 19], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 19, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [17, 11, 18], 'three_tile_probabilities': [4, 5, 2], 'three_tile_types': ['Hill', 'Pasture', 'Forest'], 'three_tile_values': [], 'all_tile_types': [0, 0, 1, 1, 1], 'all_tile_probabilities': [0, 0, 2, 4, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(19, 20), (18, 19), (19, 30)], 'three_intersection_ids': [20, 18, 30], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 20, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [11, 18, 12], 'three_tile_probabilities': [5, 2, 0], 'three_tile_types': ['Pasture', 'Forest', 'Sea'], 'three_tile_values': [8, 3, 0], 'all_tile_types': [0, 0, 1, 0, 1], 'all_tile_probabilities': [0, 0, 2, 0, 5], 'all_tile_values': [0, 1, 0, 0, 0, 0, 1, 0, 0, 0], 'three_roads': [(9, 20), (19, 20), (20, 21)], 'three_intersection_ids': [9, 19, 21], 'port_tf': True, 'port_type': '2:1 Ore'}
- Intersection Attributes: {'id': 21, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [18, 12, 19], 'three_tile_probabilities': [2, 0, 0], 'three_tile_types': ['Forest', 'Sea', 'Sea'], 'three_tile_values': [], 'all_tile_types': [0, 0, 1, 0, 0], 'all_tile_probabilities': [0, 0, 2, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(20, 21), (21, 32)], 'three_intersection_ids': [20, 32], 'port_tf': True, 'port_type': '2:1 Ore'}
- Intersection: None - [Note: this is Intersection\[22\]](#)
- Intersection Attributes: {'id': 23, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [20, 14, 21], 'three_tile_probabilities': [0, 0, 0], 'three_tile_types': ['Sea', 'Sea', 'Desert'], 'three_tile_values': [], 'all_tile_types': [0, 0, 0, 0, 0], 'all_tile_probabilities': [0, 0, 0, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(23, 24), (23, 34)], 'three_intersection_ids': [24, 34], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 24, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [14, 21, 15], 'three_tile_probabilities': [0, 0, 4], 'three_tile_types': ['Sea', 'Desert', 'Mountain'], 'three_tile_values': [0, 0, 5], 'all_tile_types': [0, 1, 0, 0, 0],

- 'all_tile_probabilities': [0, 4, 0, 0, 0], 'all_tile_values': [0, 0, 0, 1, 0, 0, 0, 0, 0, 0], 'three_roads': [(13, 24), (23, 24), (24, 25)], 'three_intersection_ids': [13, 23, 25], 'port_tf': True, 'port_type': '2:1 Wool'
- Intersection Attributes: {'id': 25, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [21, 15, 22], 'three_tile_probabilities': [0, 4, 2], 'three_tile_types': ['Desert', 'Mountain', 'Pasture'], 'three_tile_values': [], 'all_tile_types': [0, 1, 0, 0, 1], 'all_tile_probabilities': [0, 4, 0, 0, 2], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(25, 26), (24, 25), (25, 36)], 'three_intersection_ids': [26, 24, 36], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 26, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [15, 22, 16], 'three_tile_probabilities': [4, 2, 5], 'three_tile_types': ['Mountain', 'Pasture', 'Pasture'], 'three_tile_values': [5, 11, 8], 'all_tile_types': [0, 1, 0, 0, 2], 'all_tile_probabilities': [0, 4, 0, 0, 7], 'all_tile_values': [0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0], 'three_roads': [(15, 26), (25, 26), (26, 27)], 'three_intersection_ids': [15, 25, 27], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 27, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [22, 16, 23], 'three_tile_probabilities': [2, 5, 2], 'three_tile_types': ['Pasture', 'Pasture', 'Mountain'], 'three_tile_values': [], 'all_tile_types': [0, 1, 0, 0, 2], 'all_tile_probabilities': [0, 2, 0, 0, 7], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(27, 28), (26, 27), (27, 38)], 'three_intersection_ids': [28, 26, 38], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 28, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [16, 23, 17], 'three_tile_probabilities': [5, 2, 4], 'three_tile_types': ['Pasture', 'Mountain', 'Hill'], 'three_tile_values': [8, 11, 9], 'all_tile_types': [0, 1, 0, 1, 1], 'all_tile_probabilities': [0, 2, 0, 4, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0], 'three_roads': [(17, 28), (27, 28), (28, 29)], 'three_intersection_ids': [17, 27, 29], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 29, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [23, 17, 24], 'three_tile_probabilities': [2, 4, 5], 'three_tile_types': ['Mountain', 'Hill', 'Field'], 'three_tile_values': [], 'all_tile_types': [1, 1, 0, 1, 0], 'all_tile_probabilities': [5, 2, 0, 4, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(29, 30), (28, 29), (29, 40)], 'three_intersection_ids': [30, 28, 40], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 30, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [17, 24, 18], 'three_tile_probabilities': [4, 5, 2], 'three_tile_types': ['Hill', 'Field', 'Forest'], 'three_tile_values': [9, 6, 3], 'all_tile_types': [1, 0, 1, 1, 0], 'all_tile_probabilities': [5, 0, 2, 4, 0], 'all_tile_values': [0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0], 'three_roads': [(19, 30), (29, 30), (30, 31)], 'three_intersection_ids': [19, 29, 31], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 31, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [24, 18, 25], 'three_tile_probabilities': [5, 2, 3], 'three_tile_types': ['Field', 'Forest', 'Forest'], 'three_tile_values': [], 'all_tile_types': [1, 0, 2, 0, 0], 'all_tile_probabilities': [5, 0, 5, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(31, 32), (30, 31), (31, 42)], 'three_intersection_ids': [32, 30, 42], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 32, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [18, 25, 19], 'three_tile_probabilities': [2, 3, 0], 'three_tile_types': ['Forest', 'Forest', 'Sea'], 'three_tile_values': [3, 4, 0], 'all_tile_types': [0, 0, 2, 0, 0], 'all_tile_probabilities': [0, 0, 5, 0, 0], 'all_tile_values': [0, 1, 1, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(21, 32), (31, 32), (32, 33)], 'three_intersection_ids': [21, 31, 33], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 33, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [25, 19, 26], 'three_tile_probabilities': [3, 0, 0], 'three_tile_types': ['Forest', 'Sea', 'Sea'], 'three_tile_values': [], 'all_tile_types': [0, 0, 1, 0, 0], 'all_tile_probabilities': [0, 0, 3, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(32, 33), (33, 44)], 'three_intersection_ids': [32, 44], 'port_tf': True, 'port_type': '3:1'}
- Intersection Attributes: {'id': 34, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [20, 27, 21], 'three_tile_probabilities': [0, 0, 0], 'three_tile_types': ['Sea', 'Sea', 'Desert'], 'three_tile_values': [0, 0, 0], 'all_tile_types': [0, 0, 0, 0, 0], 'all_tile_probabilities': [0, 0, 0, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(23, 34), (34, 35)], 'three_intersection_ids': [23, 35], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 35, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [27, 21, 28], 'three_tile_probabilities': [0, 0, 3], 'three_tile_types': ['Sea', 'Desert', 'Field'], 'three_tile_values': [], 'all_tile_types': [1, 0, 0, 0, 0], 'all_tile_probabilities': [3, 0, 0, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(35, 36), (34, 35), (35, 46)], 'three_intersection_ids': [36, 34, 46], 'port_tf': True, 'port_type': '2:1 Wood'}
- Intersection Attributes: {'id': 36, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [21, 28, 22], 'three_tile_probabilities': [0, 3, 2], 'three_tile_types': ['Desert', 'Field', 'Pasture'], 'three_tile_values': [0, 4, 11], 'all_tile_types': [1, 0, 0, 0, 1], 'all_tile_probabilities': [3, 0, 0, 0, 2], 'all_tile_values': [0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0], 'three_roads': [(25, 36), (35, 36), (36, 37)], 'three_intersection_ids': [25, 35, 37], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 37, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [28, 22, 29], 'three_tile_probabilities': [3, 2, 3], 'three_tile_types': ['Field', 'Pasture', 'Field'], 'three_tile_values': [], 'all_tile_types': [2, 0, 0, 0, 1], 'all_tile_probabilities': [6, 0, 0, 0, 2], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(37, 38), (36, 37), (37, 48)], 'three_intersection_ids': [38, 36, 48], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 38, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [22, 29, 23], 'three_tile_probabilities': [2, 3, 2], 'three_tile_types': ['Pasture', 'Field', 'Mountain'], 'three_tile_values': [11, 10, 11], 'all_tile_types': [1, 1, 0, 0, 1], 'all_tile_probabilities': [3, 2, 0, 0, 2], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 1, 2, 0], 'three_roads': [(27, 38), (37, 38), (38, 39)], 'three_intersection_ids': [27, 37, 39], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 39, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [29, 23, 30], 'three_tile_probabilities': [3, 2, 4], 'three_tile_types': ['Field', 'Mountain', 'Hill'], 'three_tile_values': [], 'all_tile_types': [1, 1, 0, 1, 0], 'all_tile_probabilities': [3, 2, 0, 4, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(39, 40), (38, 39), (39, 50)], 'three_intersection_ids': [40, 38, 50], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 40, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [23, 30, 24], 'three_tile_probabilities': [2, 4, 5], 'three_tile_types': ['Mountain', 'Hill', 'Field'], 'three_tile_values': [11, 5, 6], 'all_tile_types': [1, 1, 0, 1, 0], 'all_tile_probabilities': [5, 2, 0, 4, 0], 'all_tile_values': [0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0], 'three_roads': [(29, 40), (39, 40), (40, 41)], 'three_intersection_ids': [29, 39, 41], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 41, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [30, 24, 31], 'three_tile_probabilities': [4, 5, 4], 'three_tile_types': ['Hill', 'Field', 'Field'], 'three_tile_values': [], 'all_tile_types': [2, 0, 0, 1, 0], 'all_tile_probabilities': [9, 0, 0, 4, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(41, 42), (40, 41), (41, 52)], 'three_intersection_ids': [42, 40, 52], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 42, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [24, 31, 25], 'three_tile_probabilities': [5, 4, 3], 'three_tile_types': ['Field', 'Field', 'Forest'], 'three_tile_values': [6, 9, 4], 'all_tile_types': [2, 0, 1, 0, 0], 'all_tile_probabilities': [9, 0, 3, 0, 0], 'all_tile_values': [0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0], 'three_roads': [(31, 42), (41, 42), (42, 43)], 'three_intersection_ids': [31, 41, 43], 'port_tf': False, 'port_type': 'Blank'}

- Intersection Attributes: {'id': 43, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [31, 25, 32], 'three_tile_probabilities': [4, 3, 0], 'three_tile_types': ['Field', 'Forest', 'Sea'], 'three_tile_values': [], 'all_tile_types': [1, 0, 1, 0, 0], 'all_tile_probabilities': [4, 0, 3, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(43, 44), (42, 43), (43, 54)], 'three_intersection_ids': [44, 42, 54], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 44, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [25, 32, 26], 'three_tile_probabilities': [3, 0, 0], 'three_tile_types': ['Forest', 'Sea', 'Sea'], 'three_tile_values': [4, 0, 0], 'all_tile_types': [0, 0, 1, 0, 0], 'all_tile_probabilities': [0, 0, 3, 0, 0], 'all_tile_values': [0, 0, 1, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(33, 44), (43, 44)], 'three_intersection_ids': [33, 43], 'port_tf': True, 'port_type': '3:1'}
- Intersection: None - [Note: this is Intersection\[45\]](#)
- Intersection Attributes: {'id': 46, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [27, 34, 28], 'three_tile_probabilities': [0, 0, 3], 'three_tile_types': ['Sea', 'Sea', 'Field'], 'three_tile_values': [0, 0, 4], 'all_tile_types': [1, 0, 0, 0, 0], 'all_tile_probabilities': [3, 0, 0, 0, 0], 'all_tile_values': [0, 0, 1, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(35, 46), (46, 47)], 'three_intersection_ids': [35, 47], 'port_tf': True, 'port_type': '2:1 Wood'}
- Intersection Attributes: {'id': 47, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [34, 28, 35], 'three_tile_probabilities': [0, 3, 5], 'three_tile_types': ['Sea', 'Field', 'Pasture'], 'three_tile_values': [], 'all_tile_types': [1, 0, 0, 0, 1], 'all_tile_probabilities': [3, 0, 0, 0, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(47, 48), (46, 47), (47, 58)], 'three_intersection_ids': [48, 46, 58], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 48, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [28, 35, 29], 'three_tile_probabilities': [3, 5, 3], 'three_tile_types': ['Field', 'Pasture', 'Field'], 'three_tile_values': [4, 6, 10], 'all_tile_types': [2, 0, 0, 0, 1], 'all_tile_probabilities': [6, 0, 0, 0, 5], 'all_tile_values': [0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0], 'three_roads': [(37, 48), (47, 48), (48, 49)], 'three_intersection_ids': [37, 47, 49], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 49, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [35, 29, 36], 'three_tile_probabilities': [5, 3, 3], 'three_tile_types': ['Pasture', 'Field', 'Forest'], 'three_tile_values': [], 'all_tile_types': [1, 0, 1, 0, 1], 'all_tile_probabilities': [3, 0, 3, 0, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(49, 50), (48, 49), (49, 60)], 'three_intersection_ids': [50, 48, 60], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 50, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [29, 36, 30], 'three_tile_probabilities': [3, 3, 4], 'three_tile_types': ['Field', 'Forest', 'Hill'], 'three_tile_values': [10, 10, 5], 'all_tile_types': [1, 0, 1, 1, 0], 'all_tile_probabilities': [3, 0, 3, 4, 0], 'all_tile_values': [0, 0, 0, 1, 0, 0, 0, 0, 2, 0, 0], 'three_roads': [(39, 50), (49, 50), (50, 51)], 'three_intersection_ids': [39, 49, 51], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 51, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [36, 30, 37], 'three_tile_probabilities': [3, 4, 1], 'three_tile_types': ['Forest', 'Hill', 'Mountain'], 'three_tile_values': [], 'all_tile_types': [0, 1, 1, 1, 0], 'all_tile_probabilities': [0, 1, 3, 4, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(51, 52), (50, 51), (51, 62)], 'three_intersection_ids': [52, 50, 62], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 52, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [30, 37, 31], 'three_tile_probabilities': [4, 1, 4], 'three_tile_types': ['Hill', 'Mountain', 'Field'], 'three_tile_values': [5, 12, 9], 'all_tile_types': [1, 1, 0, 1, 0], 'all_tile_probabilities': [4, 1, 0, 4, 0], 'all_tile_values': [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1], 'three_roads': [(41, 52), (51, 52), (52, 53)], 'three_intersection_ids': [41, 51, 53], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 53, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [37, 31, 38], 'three_tile_probabilities': [1, 4, 0], 'three_tile_types': ['Mountain', 'Field', 'Sea'], 'three_tile_values': [], 'all_tile_types': [1, 1, 0, 0, 0], 'all_tile_probabilities': [4, 1, 0, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(53, 54), (52, 53), (53, 64)], 'three_intersection_ids': [54, 52, 64], 'port_tf': True, 'port_type': '3:1'}
- Intersection Attributes: {'id': 54, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [31, 38, 32], 'three_tile_probabilities': [4, 0, 0], 'three_tile_types': ['Field', 'Sea', 'Sea'], 'three_tile_values': [9, 0, 0], 'all_tile_types': [1, 0, 0, 0, 0], 'all_tile_probabilities': [4, 0, 0, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0], 'three_roads': [(43, 54), (53, 54)], 'three_intersection_ids': [43, 53], 'port_tf': True, 'port_type': '3:1'}
- Intersection: None - [Note: this is Intersection\[55\]](#)
- Intersection: None - [Note: this is Intersection\[56\]](#)
- Intersection: None - [Note: this is Intersection\[57\]](#)
- Intersection Attributes: {'id': 58, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [34, 41, 35], 'three_tile_probabilities': [0, 0, 5], 'three_tile_types': ['Sea', 'Sea', 'Pasture'], 'three_tile_values': [0, 0, 6], 'all_tile_types': [0, 0, 0, 0, 1], 'all_tile_probabilities': [0, 0, 0, 0, 5], 'all_tile_values': [0, 0, 0, 0, 1, 0, 0, 0, 0, 0], 'three_roads': [(47, 58), (58, 59)], 'three_intersection_ids': [47, 59], 'port_tf': True, 'port_type': '2:1 Wheat'}
- Intersection Attributes: {'id': 59, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [41, 35, 42], 'three_tile_probabilities': [0, 5, 0], 'three_tile_types': ['Sea', 'Pasture', 'Sea'], 'three_tile_values': [], 'all_tile_types': [0, 0, 0, 0, 1], 'all_tile_probabilities': [0, 0, 0, 0, 5], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(59, 60), (58, 59)], 'three_intersection_ids': [60, 58], 'port_tf': True, 'port_type': '2:1 Wheat'}
- Intersection Attributes: {'id': 60, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [35, 42, 36], 'three_tile_probabilities': [0, 0, 3], 'three_tile_types': ['Pasture', 'Sea', 'Forest'], 'three_tile_values': [6, 0, 10], 'all_tile_types': [0, 0, 1, 0, 1], 'all_tile_probabilities': [5, 0, 3, 0, 5], 'all_tile_values': [0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0], 'three_roads': [(49, 60), (59, 60), (60, 61)], 'three_intersection_ids': [49, 59, 61], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 61, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [42, 36, 43], 'three_tile_probabilities': [0, 3, 0], 'three_tile_types': ['Sea', 'Forest', 'Sea'], 'three_tile_values': [], 'all_tile_types': [0, 0, 1, 0, 0], 'all_tile_probabilities': [0, 0, 3, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(61, 62), (60, 61)], 'three_intersection_ids': [62, 60], 'port_tf': True, 'port_type': '3:1'}
- Intersection Attributes: {'id': 62, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [36, 43, 37], 'three_tile_probabilities': [3, 0, 1], 'three_tile_types': ['Forest', 'Sea', 'Mountain'], 'three_tile_values': [10, 0, 12], 'all_tile_types': [0, 1, 1, 0, 0], 'all_tile_probabilities': [0, 1, 3, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1], 'three_roads': [(51, 62), (61, 62), (62, 63)], 'three_intersection_ids': [51, 61, 63], 'port_tf': True, 'port_type': '3:1'}
- Intersection Attributes: {'id': 63, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [43, 37, 44], 'three_tile_probabilities': [0, 1, 0], 'three_tile_types': ['Sea', 'Mountain', 'Sea'], 'three_tile_values': [], 'all_tile_types': [0, 1, 0, 0, 0], 'all_tile_probabilities': [0, 1, 0, 0, 0], 'all_tile_values': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'three_roads': [(63, 64), (62, 63)], 'three_intersection_ids': [64, 62], 'port_tf': False, 'port_type': 'Blank'}
- Intersection Attributes: {'id': 64, 'face': 'Blank', 'occupied_tf': False, 'color': 'Blank', 'type': 'Vertex', 'three_tile_ids': [37, 44, 38], 'three_tile_probabilities': [1, 0, 0], 'three_tile_types': ['Mountain', 'Sea', 'Sea'], 'three_tile_values': [12, 0, 0], 'all_tile_types': [0, 1, 0, 0, 0],

Road Attributes: {'id': (60, 61), 'kind': 'Blank', 'color': 'Blank', 'occupied_tf': False, 'two_tiles': [[36, 'Forest', 10], [42, 'Sea', 0]]}
 Road Attributes: {'id': (53, 64), 'kind': 'Blank', 'color': 'Blank', 'occupied_tf': False, 'two_tiles': [[37, 'Mountain', 12], [38, 'Sea', 0]]}
 Road Attributes: {'id': (63, 64), 'kind': 'Blank', 'color': 'Blank', 'occupied_tf': False, 'two_tiles': [[37, 'Mountain', 12], [44, 'Sea', 0]]}
 Road Attributes: {'id': (62, 63), 'kind': 'Blank', 'color': 'Blank', 'occupied_tf': False, 'two_tiles': [[37, 'Mountain', 12], [43, 'Sea', 0]]}

GUI data:

gui.py | _draw_tiles | Skipped tile ID: None, Pos: None
 gui.py | _draw_tiles | Skipped tile ID: None, Pos: None
 gui.py | _draw_tiles | Skipped tile ID: None, Pos: None
 gui.py | _draw_tiles | Skipped tile ID: None, Pos: None
 gui.py | _draw_tiles | Skipped tile ID: None, Pos: None
 gui.py | _draw_tiles | Skipped tile ID: None, Pos: None
 gui.py | _draw_tiles | Skipped tile ID: None, Pos: None
 gui.py | _draw_tiles | Skipped tile ID: None, Pos: None
 gui.py | _draw_tiles | Skipped tile ID: None, Pos: None
 gui.py | _draw_tiles | Rendered tile IDs: [2, 3, 4, 5, 8, 9, 10, 11, 12, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 34, 35, 36, 37, 38, 41, 42, 43, 44]
 gui_game.py | _render_scoreboard_row | Player 1: Player 1 ['0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0'] RC: [0, 0, 0, 0, 0]
 gui_game.py | _render_scoreboard_row | Player 2: Player 2 ['0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0'] RC: [0, 0, 0, 0, 0]
 gui_game.py | _render_scoreboard_row | Player 3: Player 3 ['0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0'] RC: [0, 0, 0, 0, 0]
 gui_game.py | _render_scoreboard_row | Player 4: Player 4 ['0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0'] RC: [0, 0, 0, 0, 0]
 gui_game.py | update_scoreboard | Rendered scoreboard for 4 players
 gui_human_player.py | text_buy | Active: False
 gui_human_player.py | text_trade | Active: False
 gui_human_player.py | button_buy_city | Active: False
 gui_human_player.py | button_buy_settlement | Active: False
 gui_human_player.py | button_buy_road | Active: False
 gui_human_player.py | button_buy_dcard | Active: False
 gui_human_player.py | button_twp | Active: False
 gui_human_player.py | button_twb | Active: False
 gui_human_player.py | button_next_turn2 | Active: False
 gui_human_player.py | button_end_turn | Active: False
 gui_human_player.py | button_roll_dice | Active: False
 gui_human_player.py | button_next_turn2 | Active: True
 gui_human_player.py | button_end_turn | Active: False
 gui_human_player.py | show_buttons_HP | Rendered buttons for phase: Planning, analysis_tf: False